

Robot Project Code

Formatted PDF containing the pasted robot debugging, buzzer, path plotting, PDR, motor interface, reactive navigation, and launch scripts.

Total scripts: 8

Note: code is preserved as pasted. A plain-text companion file was also generated for exact copy/paste.

1. robot_debug.py

[illegible]

2. buzzer_test.py

```
#!/usr/bin/env python3

import time
import RPi.GPIO as GPIO

BUZZER_PIN = 4      # BCM GPIO 4
FREQUENCY = 2000    # Hz

GPIO.setmode(GPIO.BCM)
GPIO.setwarnings(False)
GPIO.setup(BUZZER_PIN, GPIO.OUT)

pwm = GPIO.PWM(BUZZER_PIN, FREQUENCY)

try:
    print("PWM buzzer test on GPIO 4")
    print("Press Ctrl+C to stop.")
    print()

    while True:
        print("Buzzer ON")
        pwm.start(50)    # 50% duty cycle
        time.sleep(0.5)

        print("Buzzer OFF")
        pwm.stop()
        time.sleep(0.5)

except KeyboardInterrupt:
    print("\nStopping buzzer test...")

finally:
    pwm.stop()
    GPIO.cleanup()
    print("GPIO cleaned up.")
```

3. pi4_path_plot_receiver.py

```
#!/usr/bin/env python3

"""
Pi 4 path plotting + Pi Zero person tracking receiver.

What this does:
1. Loads the robot planned path from a CSV file.
2. Polls the Pi Zero over Wi-Fi for person/PDR coordinates.
3. Stores the received person coordinates.
4. When stopped, saves a plot showing:
   - planned robot path
   - person tracked path
   - start/end markers
5. Saves received person tracking data to a CSV file.

Expected Pi Zero endpoint:
http://PI_ZERO_IP:8000/position

Expected JSON from Pi Zero:
{
    "x": 0.0,
    "y": 0.0,
    "steps_total": 0,
    "direction": "N",
    "moving": false,
    "t": 0.0
}
"""

import csv
import json
import os
import time
import threading
import urllib.request
import urllib.error

import pandas as pd

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

# =====
# USER CONFIGURATION
# =====

# Planned robot path CSV from mapping/path planning
PLANNED_PATH_CSV = "/tmp/planned_path.csv"

# Output files
OUTPUT_IMAGE = "/home/pi/ros2_ws/src/images/path_plot_with_person3.png"
PERSON_LOG_CSV = "/home/pi/ros2_ws/src/images/person_tracking_log3.csv"

# Pi Zero server info
# CHANGE THIS to the Pi Zero's actual IP address
PI_ZERO_IP = "10.49.245.84"
PI_ZERO_PORT = 8000
PI_ZERO_POSITION_ENDPOINT = f"http://{PI_ZERO_IP}:{PI_ZERO_PORT}/position"

# How often Pi 4 asks Pi Zero for person tracking data
POLL_PERIOD_SEC = 0.25

# Optional coordinate scaling/offsets
# Use these if the Pi Zero PDR coordinate frame needs to be aligned to the map.
PERSON_X_SCALE = 1.0
PERSON_Y_SCALE = 1.0
PERSON_X_OFFSET = 0.0
PERSON_Y_OFFSET = 0.0

# If the person path is mirrored, change these to True
FLIP_PERSON_X = False
FLIP_PERSON_Y = False

# If the person path is rotated relative to the planned path,
# use this. Try 0, 90, -90, or 180.
PERSON_ROTATION_DEG = 0.0

# =====
# STOP CONTROL
# =====

stop_event = threading.Event()

def wait_for_stop():
    input("\nPress Enter to stop recording and save the final plot...\n")
    stop_event.set()

# =====
# HELPERS
```

```

# =====
def transform_person_xy(x, y):
    """
    Apply simple transform to align Pi Zero PDR coordinates with the plotted map.

    Order:
    1. optional flip x/y
    2. rotation
    3. scale
    4. offset
    """
    import math

    x = float(x)
    y = float(y)

    if FLIP_PERSON_X:
        x = -x

    if FLIP_PERSON_Y:
        y = -y

    theta = math.radians(PERSON_ROTATION_DEG)

    xr = x * math.cos(theta) - y * math.sin(theta)
    yr = x * math.sin(theta) + y * math.cos(theta)

    xr = PERSON_X_SCALE * xr + PERSON_X_OFFSET
    yr = PERSON_Y_SCALE * yr + PERSON_Y_OFFSET

    return xr, yr

def load_planned_path(csv_path):
    """
    Load planned path CSV.

    Supports:
    - CSV with x,y columns
    - CSV with any first two numeric columns
    """
    if not os.path.exists(csv_path):
        print(f"WARNING: Planned path CSV not found: {csv_path}")
        print("The final plot will only show the person path.")
        return None, None, None

    df = pd.read_csv(csv_path)

    if len(df.columns) < 2:
        raise ValueError("Planned path CSV must have at least two columns.")

    cols = df.columns.tolist()

    x_col = "x" if "x" in cols else cols[0]
    y_col = "y" if "y" in cols else cols[1]

    print(f"Loaded planned path from: {csv_path}")
    print(f"Using planned path columns: {x_col}, {y_col}")
    print(f"Planned path points: {len(df)}")

    return df, x_col, y_col

def fetch_pi_zero_position():
    """
    Poll Pi Zero /position endpoint and return parsed JSON.
    """
    with urllib.request.urlopen(PI_ZERO_POSITION_ENDPOINT, timeout=2.0) as response:
        raw = response.read().decode("utf-8")
        data = json.loads(raw)
        return data

def save_person_log(person_records, output_csv):
    """
    Save received Pi Zero tracking data to CSV.
    """
    with open(output_csv, "w", newline="") as f:
        writer = csv.writer(f)

        writer.writerow([
            "pi4_time_s",
            "pi_zero_time_s",
            "x_raw",
            "y_raw",
            "x_plot",
            "y_plot",
            "steps_total",
            "direction",
            "moving",
            "last_step_time",
        ])

        for r in person_records:
            writer.writerow([
                r.get("pi4_time_s", ""),

```

```

        r.get("pi_zero_time_s", ""),
        r.get("x_raw", ""),
        r.get("y_raw", ""),
        r.get("x_plot", ""),
        r.get("y_plot", ""),
        r.get("steps_total", ""),
        r.get("direction", ""),
        r.get("moving", ""),
        r.get("last_step_time", ""),
    ])

    print(f"Person tracking log saved to: {output_csv}")

def save_final_plot(planned_df, x_col, y_col, person_records, output_image):
    """
    Save final plot with planned path and person path.
    """
    plt.figure(figsize=(9, 9))

    # -----
    # Plot planned robot path
    # -----
    if planned_df is not None and len(planned_df) > 0:
        plt.plot(
            planned_df[x_col],
            planned_df[y_col],
            "b-",
            alpha=0.6,
            linewidth=2,
            label="Planned Robot Path"
        )

        plt.scatter(
            planned_df[x_col],
            planned_df[y_col],
            c=range(len(planned_df)),
            cmap="viridis",
            s=12,
            alpha=0.8,
            label="Planned Path Points"
        )

        plt.plot(
            planned_df[x_col].iloc[0],
            planned_df[y_col].iloc[0],
            "go",
            markersize=10,
            label="Robot Path Start"
        )

        plt.plot(
            planned_df[x_col].iloc[-1],
            planned_df[y_col].iloc[-1],
            "ro",
            markersize=10,
            label="Robot Path End"
        )

    # -----
    # Plot person/PDR path
    # -----
    if len(person_records) > 0:
        person_x = [r["x_plot"] for r in person_records]
        person_y = [r["y_plot"] for r in person_records]

        plt.plot(
            person_x,
            person_y,
            "k-",
            linewidth=2,
            alpha=0.8,
            label="Person Tracked Path"
        )

        plt.scatter(
            person_x,
            person_y,
            c=range(len(person_x)),
            cmap="plasma",
            s=18,
            alpha=0.9,
            label="Person Tracking Points"
        )

        plt.plot(
            person_x[0],
            person_y[0],
            marker="o",
            markersize=10,
            color="lime",
            label="Person Start"
        )

        plt.plot(
            person_x[-1],
            person_y[-1],

```



```

        marker="x",
        markersize=12,
        color="red",
        label="Person End"
    )

    # Label every few points so the path direction is visible
    label_every = max(1, len(person_records) // 10)

    for i, r in enumerate(person_records):
        if i % label_every == 0 or i == len(person_records) - 1:
            label = str(r.get("direction", ""))
            plt.text(
                r["x_plot"],
                r["y_plot"],
                label,
                fontsize=8
            )
        else:
            print("WARNING: No person records were received. Plot will only show planned path.")

    # -----
    # Plot formatting
    # -----
    plt.title("Planned Robot Path with Pi Zero Person Tracking Overlay")
    plt.xlabel("X position")
    plt.ylabel("Y position")
    plt.grid(True)
    plt.axis("equal")
    plt.legend(loc="best")

    plt.savefig(output_image, dpi=150, bbox_inches="tight")
    plt.close()

    print(f"Final plot saved to: {output_image}")

# =====
# MAIN
# =====

def main():
    print("=====")
    print(" Pi 4 Planned Path + Person Tracking Plotter")
    print("=====")
    print(f"Planned path CSV: {PLANNED_PATH_CSV}")
    print(f"Pi Zero endpoint: {PI_ZERO_POSITION_ENDPOINT}")
    print(f"Output image: {OUTPUT_IMAGE}")
    print(f"Person log CSV: {PERSON_LOG_CSV}")
    print()

    planned_df, x_col, y_col = load_planned_path(PLANNED_PATH_CSV)

    person_records = []

    stop_thread = threading.Thread(target=wait_for_stop, daemon=True)
    stop_thread.start()

    print("Starting Pi Zero polling.")
    print("Walk with the Pi Zero/BN0055 now.")
    print("Press Enter when finished to save the plot.")
    print()

    start_time = time.perf_counter()
    last_good_time = None
    poll_count = 0
    error_count = 0

    while not stop_event.is_set():
        loop_start = time.perf_counter()
        pi4_time = loop_start - start_time

        try:
            data = fetch_pi_zero_position()
            poll_count += 1
            last_good_time = pi4_time

            # Pull position from Pi Zero JSON
            x_raw = float(data.get("x", 0.0))
            y_raw = float(data.get("y", 0.0))

            x_plot, y_plot = transform_person_xy(x_raw, y_raw)

            record = {
                "pi4_time_s": round(pi4_time, 3),
                "pi_zero_time_s": data.get("t", data.get("time_s", "")),
                "x_raw": x_raw,
                "y_raw": y_raw,
                "x_plot": x_plot,
                "y_plot": y_plot,
                "steps_total": data.get("steps_total", data.get("steps", "")),
                "direction": data.get("direction", data.get("heading_used_dir", "")),
                "moving": data.get("moving", ""),
                "last_step_time": data.get("last_step_time", ""),
            }
            person_records.append(record)

```

```

        print(
            f"RX {poll_count:04d} | "
            f"x={x_raw: .2f}, y={y_raw: .2f} "
            f"-> plot=({x_plot: .2f}, {y_plot: .2f}) | "
            f"steps={record['steps_total']} | "
            f"dir={record['direction']} | "
            f"moving={record['moving']}"
        )

    except urllib.error.URLError as e:
        error_count += 1
        print(f"Connection error #{error_count}: Could not reach Pi Zero: {e}")

    except json.JSONDecodeError as e:
        error_count += 1
        print(f"JSON error #{error_count}: Pi Zero returned invalid JSON: {e}")

    except Exception as e:
        error_count += 1
        print(f"Error #{error_count}: {e}")

    sleep_time = POLL_PERIOD_SEC - (time.perf_counter() - loop_start)
    if sleep_time > 0:
        time.sleep(sleep_time)

    print()
    print("Stopping receiver.")
    print(f"Total good polls: {poll_count}")
    print(f"Total errors: {error_count}")

    if last_good_time is not None:
        print(f"Last good packet: {last_good_time:.2f} s after start")
    else:
        print("No successful packets were received from Pi Zero.")

    save_person_log(person_records, PERSON_LOG_CSV)
    save_final_plot(planned_df, x_col, y_col, person_records, OUTPUT_IMAGE)

    print()
    print("Done.")
    print(f"Copy/view this image from your computer:")
    print(f" {OUTPUT_IMAGE}")

if __name__ == "__main__":
    main()

```

4. pi_zero_pdr_server.py

```
#!/usr/bin/env python3
"""
BN0055 PDR prototype + Wi-Fi live position server.

Features:
- acceleration-magnitude step signal
- rolling average smoothing
- slow baseline removal
- event-based peak capture
- cooldown between steps
- minimum absolute peak requirement
- press Enter to start
- press Enter again to stop early
- starting facing direction becomes "north"
- heading debug plot
- 90-degree discrete heading bins: N, E, S, W
- direction hysteresis / direction locking
- vertical IMU mount support using a body-forward sensor axis
- fixed 0.61 m step length, about 2 feet
- live Wi-Fi server so Pi 4 can receive x/y/person tracking data

Pi 4 should poll:
    http://PI_ZERO_IP:8000/position

Useful endpoints:
    /position
    /status
    /reset

Outputs:
1. XY trajectory plot
2. ax, ay, az vs time plot
3. step_signal vs time with detected steps marked
4. heading debug plot
5. trajectory CSV with heading data
"""

import csv
import json
import math
import time
import threading
from collections import deque
from http.server import BaseHTTPRequestHandler, ThreadingHTTPServer
from urllib.parse import urlparse

import board
import adafruit_bno055
import numpy as np

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

# -----
# WIFI SERVER SETTINGS
# -----
SERVER_HOST = "0.0.0.0"
SERVER_PORT = 8000

# -----
# USER SETTINGS
# -----
SAMPLE_HZ = 50.0
DT_TARGET = 1.0 / SAMPLE_HZ
RUN_TIME_SEC = 1000.0

# -----
# MODE SELECTION
# -----
# Change this to compare modes:
# "IMUPLUS" = gyro + accel fusion, no magnetometer.
# "NDOF"    = gyro + accel + magnetometer fusion.
BNO_MODE_NAME = "IMUPLUS"

if BNO_MODE_NAME == "IMUPLUS":
    BNO_MODE = adafruit_bno055.IMUPLUS_MODE
elif BNO_MODE_NAME == "NDOF":
    BNO_MODE = adafruit_bno055.NDOF_MODE
else:
    raise ValueError("BNO_MODE_NAME must be either 'IMUPLUS' or 'NDOF'")

# -----
# OUTPUT FILES
# -----
OUTPUT_DIR = "/home/kaelembent/Final_Project/images"
OUTPUT_TAG = f"vertical_{BNO_MODE_NAME.lower()}_cardinal_locked_maze"

OUTPUT_PNG_XY = f"{OUTPUT_DIR}/pdr_track_xy_{OUTPUT_TAG}.png"
OUTPUT_PNG_ACC = f"{OUTPUT_DIR}/pdr_acc_xyz_{OUTPUT_TAG}.png"
```

```

OUTPUT_PNG_SIGNAL = f"{OUTPUT_DIR}/pdr_step_signal_{OUTPUT_TAG}.png"
OUTPUT_PNG_HEADING = f"{OUTPUT_DIR}/pdr_heading_debug_{OUTPUT_TAG}.png"
OUTPUT_CSV = f"{OUTPUT_DIR}/pdr_track_{OUTPUT_TAG}.csv"

# -----
# IMU MOUNTING SETTING
# -----
# Since the BN0055 is mounted vertically on the body, heading should be computed
# from the sensor axis that points in the person's walking direction.
#
# Try "+Z" first if the board face points forward away from your body.
# Try "-Z" if the board face points into your body.
#
# Other options:
#   "+X", "-X", "+Y", "-Y", "+Z", "-Z"
BODY_FORWARD_AXIS = "+Z"

# -----
# PDR SETTINGS
# -----

# heading smoothing
HEADING_LPF_ALPHA = 0.25

# Optional heading correction.
# Use this if the whole path is rotated by a fixed amount.
# Try 0, 90, -90, or 180 if the axes look wrong.
HEADING_OFFSET_DEG = 0.0
HEADING_OFFSET_RAD = math.radians(HEADING_OFFSET_DEG)

# If right turns become left turns, flip this.
USE_FLIPPED_HEADING_SIGN = True

# If your x-direction is still mirrored after trying USE_FLIPPED_HEADING_SIGN,
# set this to True.
FLIP_X_UPDATE = False

# 90-degree discrete heading mode
USE_CARDINAL_HEADING = True

# Direction locking / hysteresis
# This prevents N/E/N/E flickering when heading is near a boundary.
DIRECTION_SWITCH_TO_NEW_DEG = 30.0
DIRECTION_LEAVE_CURRENT_DEG = 55.0
DIRECTION_CONFIRM_STEPS = 2

# step-signal construction
ROLLING_WINDOW = 5
BASELINE_WINDOW = 60

# event-based detection thresholds
STEP_EVENT_START = 2.2
STEP_EVENT_END = 0.5
MIN_STEP_PROMINENCE = 2.0
MIN_STEP_INTERVAL = 0.45

# absolute peak threshold
MIN_STEP_PEAK = 5.0

# fixed step length
FIXED_STEP_LEN_M = 0.61 # about 2 feet
MIN_STEP_LEN = 0.0
MAX_STEP_LEN = 1.10

# plot options
LABEL_STEP_TIMES = True

# moving status timeout
MOVING_TIMEOUT_SEC = 1.5

# -----
# Stop control
# -----
stop_event = threading.Event()

def wait_for_stop_input():
    input("\nPress Enter again at any time to stop recording early...\n")
    stop_event.set()

# -----
# Shared live state for Wi-Fi server
# -----
state_lock = threading.Lock()

live_state = {
    "ok": True,
    "running": False,
    "x": 0.0,
    "y": 0.0,
    "steps_total": 0,
    "steps": 0,
    "direction": "N",

```

```

    "moving": False,
    "t": 0.0,
    "last_step_time": None,
    "heading_deg": 0.0,
    "heading_used_deg": 0.0,
    "instant_direction": "N",
    "instant_heading_deg": 0.0,
    "axis_heading_deg": 0.0,
    "original_yaw_deg": 0.0,
    "cal_sys": 0,
    "cal_gyro": 0,
    "cal_acc": 0,
    "cal_mag": 0,
    "mode": BNO_MODE_NAME,
    "body_forward_axis": BODY_FORWARD_AXIS,
}

def reset_live_state():
    with state_lock:
        live_state["x"] = 0.0
        live_state["y"] = 0.0
        live_state["steps_total"] = 0
        live_state["steps"] = 0
        live_state["direction"] = "N"
        live_state["moving"] = False
        live_state["t"] = 0.0
        live_state["last_step_time"] = None
        live_state["heading_deg"] = 0.0
        live_state["heading_used_deg"] = 0.0
        live_state["instant_direction"] = "N"
        live_state["instant_heading_deg"] = 0.0
        live_state["axis_heading_deg"] = 0.0
        live_state["original_yaw_deg"] = 0.0

class PositionHandler(BaseHTTPRequestHandler):
    def do_GET(self):
        parsed = urlparse(self.path)
        path = parsed.path

        if path == "/position" or path == "/status":
            with state_lock:
                data = dict(live_state)
            self.send_json(data)
            return

        if path == "/reset":
            reset_live_state()
            self.send_json({"ok": True, "message": "Pi Zero tracking state reset"})
            return

        self.send_json({
            "ok": False,
            "error": "Valid endpoints are /position, /status, /reset"
        })

    def send_json(self, data):
        payload = json.dumps(data).encode("utf-8")

        self.send_response(200)
        self.send_header("Content-Type", "application/json")
        self.send_header("Access-Control-Allow-Origin", "")
        self.send_header("Content-Length", str(len(payload)))
        self.end_headers()
        self.wfile.write(payload)

    def log_message(self, format, *args):
        return

def start_wifi_server():
    server = ThreadingHTTPServer((SERVER_HOST, SERVER_PORT), PositionHandler)
    print(f"Pi Zero Wi-Fi server running on http://0.0.0.0:{SERVER_PORT}")
    print("Pi 4 should poll:")
    print(f"  http://<PI_ZERO_IP>:{SERVER_PORT}/position")
    print("Available endpoints:")
    print("  /position")
    print("  /status")
    print("  /reset")
    print()
    server.serve_forever()

# -----
# Helpers
# -----
def get_axis_vector(axis_name):
    """
    Return a unit vector for the selected sensor/body axis.
    """
    axes = {
        "+X": np.array([1.0, 0.0, 0.0]),
        "-X": np.array([-1.0, 0.0, 0.0]),
        "+Y": np.array([0.0, 1.0, 0.0]),
        "-Y": np.array([0.0, -1.0, 0.0]),
        "+Z": np.array([0.0, 0.0, 1.0]),
    }

```

```

    "-Z": np.array([0.0, 0.0, -1.0]),
}

if axis_name not in axes:
    raise ValueError("BODY_FORWARD_AXIS must be one of +X, -X, +Y, -Y, +Z, -Z")

return axes[axis_name]

def quat_to_rotation_matrix(q):
    """
    Convert BN0055 quaternion (w, x, y, z) to a rotation matrix.
    """
    w, x, y, z = q

    return np.array([
        [
            1.0 - 2.0 * (y * y + z * z),
            2.0 * (x * y - w * z),
            2.0 * (x * z + w * y),
        ],
        [
            2.0 * (x * y + w * z),
            1.0 - 2.0 * (x * x + z * z),
            2.0 * (y * z - w * x),
        ],
        [
            2.0 * (x * z - w * y),
            2.0 * (y * z + w * x),
            1.0 - 2.0 * (x * x + y * y),
        ]
    ])

def quat_to_heading_from_forward_axis(q, forward_axis_name):
    """
    Compute walking heading from the sensor axis that points forward on the body.

    This is better than plain yaw when the IMU is mounted vertically.
    It rotates the chosen sensor-forward axis into the world frame, projects that
    vector onto the horizontal XY plane, and calculates the angle of that vector.
    """
    q = np.array(q, dtype=float)

    norm = np.linalg.norm(q)
    if norm < 1e-6:
        return None

    q = q / norm

    forward_sensor = get_axis_vector(forward_axis_name)
    R = quat_to_rotation_matrix(q)

    forward_world = R @ forward_sensor

    fx = forward_world[0]
    fy = forward_world[1]

    if abs(fx) < 1e-6 and abs(fy) < 1e-6:
        return None

    return math.atan2(fy, fx)

def quat_to_yaw(q):
    """
    Original yaw calculation, kept for optional debugging/reference.
    The main code now uses quat_to_heading_from_forward_axis().
    """
    w, x, y, z = q
    siny_cosp = 2.0 * (w * z + x * y)
    cosy_cosp = 1.0 - 2.0 * (y * y + z * z)
    return math.atan2(siny_cosp, cosy_cosp)

def wrap_angle(a):
    """Wrap angle to [-pi, pi]."""
    while a > math.pi:
        a -= 2.0 * math.pi
    while a < -math.pi:
        a += 2.0 * math.pi
    return a

def wrap_deg(deg):
    """Wrap degrees to [-180, 180]."""
    return (deg + 180.0) % 360.0 - 180.0

def angle_lerp(prev, new, alpha):
    """Low-pass filter angle while handling wraparound."""
    diff = wrap_angle(new - prev)
    return wrap_angle(prev + alpha * diff)

def clamp(x, lo, hi):
    return max(lo, min(hi, x))

```

```

def get_current_heading(sensor):
    """
    Read current BNO055 quaternion and compute heading using the selected
    body-forward sensor axis.
    """
    q = sensor.quaternion
    if q is None or None in q:
        return None

    return quat_to_heading_from_forward_axis(q, BODY_FORWARD_AXIS)

CARDINAL_DEGS = {
    "N": 0.0,
    "E": 90.0,
    "S": 180.0,
    "W": -90.0,
}

def angle_error_deg(a, b):
    """
    Smallest signed difference from b to a in degrees.
    Returns value in [-180, 180).
    """
    return (a - b + 180.0) % 360.0 - 180.0

def angle_error_abs_deg(a, b):
    return abs(angle_error_deg(a, b))

def nearest_cardinal_dir(heading_deg):
    heading_deg = wrap_deg(heading_deg)
    return min(
        CARDINAL_DEGS.keys(),
        key=lambda d: angle_error_abs_deg(heading_deg, CARDINAL_DEGS[d])
    )

def cardinal_dir_to_rad(direction):
    return math.radians(CARDINAL_DEGS[direction])

def quantize_heading_cardinal(heading_rad):
    """
    Instant quantization to one of four cardinal directions.
    This is still logged for debugging, but the actual PDR update uses
    the direction-locked accepted direction.
    """
    deg = math.degrees(heading_rad)
    deg = wrap_deg(deg)
    direction = nearest_cardinal_dir(deg)
    return cardinal_dir_to_rad(direction), CARDINAL_DEGS[direction], direction

# -----
# Start Wi-Fi server before sensor setup
# -----
server_thread = threading.Thread(target=start_wifi_server, daemon=True)
server_thread.start()

# -----
# Sensor setup
# -----
i2c = board.I2C()
sensor = adafruit_bno055.BNO055_I2C(i2c)

time.sleep(1.0)
sensor.mode = BNO_MODE
time.sleep(0.1)

print("BNO055 started")
print("Selected mode:", BNO_MODE_NAME)
print("Sensor mode value:", sensor.mode)
print("Body forward axis:", BODY_FORWARD_AXIS)
print("Calibration status is (sys, gyro, accel, mag)")
print("Current calibration:", sensor.calibration_status)
print()

# -----
# Wait for user to start
# -----
print("Mount the IMU vertically in the same orientation you will use while walking.")
print("Face the direction you want to define as NORTH.")
print("Hold the IMU still for a moment.")
input("Press Enter to start recording PDR data and serving live position...\n")

# Capture the starting heading and define it as north.
time.sleep(0.2)

north_heading = None
for _ in range(30):
    north_heading = get_current_heading(sensor)
    if north_heading is not None:

```

```

        break
    time.sleep(0.05)

if north_heading is None:
    raise RuntimeError("Could not read initial heading for north reference.")

print(f"Reference north heading captured: {north_heading:.3f} rad")
print("This starting facing direction is now treated as north.")
print(f"Heading offset: {HEADING_OFFSET_DEG:.1f} deg")
print(f"Flipped heading sign: {USE_FLIPPED_HEADING_SIGN}")
print(f"Cardinal heading mode: {USE_CARDINAL_HEADING}")
print(f"Flip X update: {FLIP_X_UPDATE}")
print(f"Direction switch to new threshold: {DIRECTION_SWITCH_TO_NEW_DEG:.1f} deg")
print(f"Direction leave current threshold: {DIRECTION_LEAVE_CURRENT_DEG:.1f} deg")
print(f"Direction confirm steps: {DIRECTION_CONFIRM_STEPS}")
print(f"Fixed step length: {FIXED_STEP_LEN_M:.2f} m")
print()

with state_lock:
    live_state["running"] = True
    live_state["mode"] = BNO_MODE_NAME
    live_state["body_forward_axis"] = BODY_FORWARD_AXIS

# Start background listener for stop.
stop_thread = threading.Thread(target=wait_for_stop_input, daemon=True)
stop_thread.start()

# -----
# State
# -----
x = 0.0
y = 0.0
steps_total = 0

times = []
xs = []
ys = []

step_events = []

# raw accel logs
acc_t = []
acc_x = []
acc_y = []
acc_z = []

# step-signal logs
sig_t = []
sig_raw_log = []
sig_filt_log = []
baseline_log = []
step_signal_log = []

# heading logs
heading_t = []
raw_heading_deg_log = []
rel_heading_deg_log = []
smooth_heading_deg_log = []

# optional original yaw logs
original_yaw_deg_log = []

# instant quantized heading logs
quant_heading_deg_log = []
quant_heading_dir_log = []

# accepted/locked heading logs
accepted_heading_deg_log = []
accepted_heading_dir_log = []

# calibration logs
cal_sys_log = []
cal_gyro_log = []
cal_acc_log = []
cal_mag_log = []

# buffers
sig_window = deque(maxlen=ROLLING_WINDOW)
baseline_window = deque(maxlen=BASELINE_WINDOW)

last_step_time = -999.0
prev_heading = None

# Direction locking state
accepted_heading_dir = "N"
pending_heading_dir = None
pending_heading_count = 0

# event detector state
in_step_event = False
event_peak_val = -1e9
event_peak_time = None
event_start_valley = 0.0

start_time = time.perf_counter()
prev_time = start_time

```



```

# -----
# Main loop
# -----
while True:
    now = time.perf_counter()
    elapsed = now - start_time

    if elapsed >= RUN_TIME_SEC:
        print("\nReached maximum run time.")
        break

    if stop_event.is_set():
        print("\nStopped early by user.")
        break

    dt = now - prev_time
    prev_time = now

    lin = sensor.linear_acceleration
    quat = sensor.quaternion
    gyro = sensor.gyro
    cal = sensor.calibration_status

    if (
        not lin or not quat or not gyro
        or None in lin or None in quat or None in gyro
    ):
        time.sleep(0.005)
        continue

    q = np.array(quat, dtype=float)
    if np.linalg.norm(q) < 1e-6:
        time.sleep(0.005)
        continue

    a = np.array(lin, dtype=float)

    # log raw acceleration
    acc_t.append(elapsed)
    acc_x.append(a[0])
    acc_y.append(a[1])
    acc_z.append(a[2])

    # -----
    # Heading relative to starting north direction
    # -----
    raw_heading = quat_to_heading_from_forward_axis(q, BODY_FORWARD_AXIS)

    if raw_heading is None:
        time.sleep(0.005)
        continue

    # Optional original yaw for comparison/debugging.
    original_yaw = quat_to_yaw(q)

    if USE_FLIPPED_HEADING_SIGN:
        raw_heading_relative = wrap_angle(north_heading - raw_heading + HEADING_OFFSET_RAD)
    else:
        raw_heading_relative = wrap_angle(raw_heading - north_heading + HEADING_OFFSET_RAD)

    if prev_heading is None:
        heading = raw_heading_relative
    else:
        heading = angle_lerp(prev_heading, raw_heading_relative, HEADING_LPF_ALPHA)

    prev_heading = heading

    # Instant quantization, used only for debugging now.
    heading_q, heading_q_deg, heading_q_dir = quantize_heading_cardinal(heading)

    # log heading for debugging
    heading_t.append(elapsed)
    raw_heading_deg_log.append(math.degrees(raw_heading))
    rel_heading_deg_log.append(math.degrees(raw_heading_relative))
    smooth_heading_deg_log.append(math.degrees(heading))
    original_yaw_deg_log.append(math.degrees(original_yaw))
    quant_heading_deg_log.append(heading_q_deg)
    quant_heading_dir_log.append(heading_q_dir)
    accepted_heading_deg_log.append(CARDINAL_DEGS[accepted_heading_dir])
    accepted_heading_dir_log.append(accepted_heading_dir)

    cal_sys_log.append(cal[0])
    cal_gyro_log.append(cal[1])
    cal_acc_log.append(cal[2])
    cal_mag_log.append(cal[3])

    # -----
    # Step signal
    # -----
    acc_mag = float(np.linalg.norm(a))
    sig_window.append(acc_mag)
    sig_filt = float(np.mean(sig_window))

    baseline_window.append(sig_filt)
    baseline = float(np.mean(baseline_window))

    step_signal = sig_filt - baseline

```

```

# log step-signal info
sig_t.append(elapsed)
sig_raw_log.append(acc_mag)
sig_filt_log.append(sig_filt)
baseline_log.append(baseline)
step_signal_log.append(step_signal)

# -----
# Event-based step detection
# -----
if (not in_step_event) and (step_signal > STEP_EVENT_START):
    in_step_event = True
    event_peak_val = step_signal
    event_peak_time = elapsed
    event_start_valley = (
        min(step_signal_log[-10:])
        if len(step_signal_log) >= 10
        else step_signal
    )

if in_step_event:
    if step_signal > event_peak_val:
        event_peak_val = step_signal
        event_peak_time = elapsed

    if step_signal < STEP_EVENT_END:
        prominence = event_peak_val - event_start_valley
        cooldown_ok = (event_peak_time - last_step_time) >= MIN_STEP_INTERVAL
        prominence_ok = prominence >= MIN_STEP_PROMINENCE
        peak_ok = event_peak_val >= MIN_STEP_PEAK

        if cooldown_ok and prominence_ok and peak_ok:
            t_step = event_peak_time

            step_len = FIXED_STEP_LEN_M
            step_len = clamp(step_len, MIN_STEP_LEN, MAX_STEP_LEN)

            continuous_heading_deg = math.degrees(heading)
            nearest_dir = nearest_cardinal_dir(continuous_heading_deg)

            # -----
            # Direction hysteresis / direction locking
            # -----
            if USE_CARDINAL_HEADING:
                if nearest_dir == accepted_heading_dir:
                    pending_heading_dir = None
                    pending_heading_count = 0
                else:
                    err_to_new = angle_error_abs_deg(
                        continuous_heading_deg,
                        CARDINAL_DEGS[nearest_dir]
                    )
                    err_from_current = angle_error_abs_deg(
                        continuous_heading_deg,
                        CARDINAL_DEGS[accepted_heading_dir]
                    )

                    should_consider_switch = (
                        err_to_new <= DIRECTION_SWITCH_TO_NEW_DEG
                        and err_from_current >= DIRECTION_LEAVE_CURRENT_DEG
                    )

                    if should_consider_switch:
                        if pending_heading_dir == nearest_dir:
                            pending_heading_count += 1
                        else:
                            pending_heading_dir = nearest_dir
                            pending_heading_count = 1

                        if pending_heading_count >= DIRECTION_CONFIRM_STEPS:
                            print(
                                f"DIRECTION SWITCH: {accepted_heading_dir} -> {nearest_dir} "
                                f"at t={t_step:.2f}s, heading={continuous_heading_deg:.1f} deg"
                            )
                            accepted_heading_dir = nearest_dir
                            pending_heading_dir = None
                            pending_heading_count = 0
                        else:
                            pending_heading_dir = None
                            pending_heading_count = 0

                    heading_used_dir = accepted_heading_dir
                    heading_used_deg = CARDINAL_DEGS[accepted_heading_dir]
                    heading_used = cardinal_dir_to_rad(accepted_heading_dir)
            else:
                heading_used = heading
                heading_used_deg = math.degrees(heading)
                heading_used_dir = "continuous"

            # PDR update.
            # heading = 0 means movement in +Y direction.
            dx = step_len * math.sin(heading_used)
            dy = step_len * math.cos(heading_used)

            if FLIP_X_UPDATE:
                dx = -dx

```

```

x += dx
y += dy
steps_total += 1

last_step_time = t_step

step_events.append({
    "t": t_step,
    "signal_peak": event_peak_val,
    "prominence": prominence,
    "step_len": step_len,
    "raw_heading_deg": math.degrees(raw_heading),
    "original_yaw_deg": math.degrees(original_yaw),
    "heading_rad": heading,
    "heading_deg": continuous_heading_deg,
    "instant_heading_dir": heading_q_dir,
    "instant_heading_deg": heading_q_deg,
    "heading_used_rad": heading_used,
    "heading_used_deg": heading_used_deg,
    "heading_used_dir": heading_used_dir,
    "pending_heading_dir": pending_heading_dir if pending_heading_dir else "",
    "pending_heading_count": pending_heading_count,
    "x": x,
    "y": y,
    "dx": dx,
    "dy": dy,
    "steps_total": steps_total,
    "cal_sys": cal[0],
    "cal_gyro": cal[1],
    "cal_acc": cal[2],
    "cal_mag": cal[3],
})

print(
    f"STEP {steps_total:03d} @ {t_step:5.2f}s | "
    f"peak={event_peak_val: .3f} | prom={prominence: .3f} | "
    f"len={step_len: .3f} m | "
    f"axis_heading={math.degrees(raw_heading): .1f} deg | "
    f"rel_heading={continuous_heading_deg: .1f} deg | "
    f"instant={heading_q_dir} | "
    f"used={heading_used_dir}({heading_used_deg: .0f} deg) | "
    f"pending={pending_heading_dir}({pending_heading_count} | "
    f"dx={dx: .3f}, dy={dy: .3f} | "
    f"pos={x: .3f}, {y: .3f}) | cal={cal}"
)

else:
    print(
        f"REJECT @ {event_peak_time:5.2f}s | "
        f"peak={event_peak_val: .3f} | prom={prominence: .3f} | "
        f"cooldown_ok={cooldown_ok} | peak_ok={peak_ok} | prom_ok={prominence_ok}"
    )

in_step_event = False
event_peak_val = -1e9
event_peak_time = None
event_start_valley = 0.0

times.append(elapsed)
xs.append(x)
ys.append(y)

# -----
# Update Wi-Fi live state for Pi 4
# -----
moving = (elapsed - last_step_time) <= MOVING_TIMEOUT_SEC if last_step_time >= 0 else False

with state_lock:
    live_state["ok"] = True
    live_state["running"] = True
    live_state["x"] = round(float(x), 4)
    live_state["y"] = round(float(y), 4)
    live_state["steps_total"] = int(steps_total)
    live_state["steps"] = int(steps_total)
    live_state["direction"] = str(accepted_heading_dir)
    live_state["moving"] = bool(moving)
    live_state["t"] = round(float(elapsed), 3)
    live_state["last_step_time"] = None if last_step_time < 0 else round(float(last_step_time), 3)
    live_state["heading_deg"] = round(float(math.degrees(heading)), 2)
    live_state["heading_used_deg"] = round(float(CARDINAL_DEGS[accepted_heading_dir]), 2)
    live_state["instant_direction"] = str(heading_q_dir)
    live_state["instant_heading_deg"] = round(float(heading_q_deg), 2)
    live_state["axis_heading_deg"] = round(float(math.degrees(raw_heading)), 2)
    live_state["original_yaw_deg"] = round(float(math.degrees(original_yaw)), 2)
    live_state["cal_sys"] = int(cal[0])
    live_state["cal_gyro"] = int(cal[1])
    live_state["cal_acc"] = int(cal[2])
    live_state["cal_mag"] = int(cal[3])
    live_state["mode"] = BNO_MODE_NAME
    live_state["body_forward_axis"] = BODY_FORWARD_AXIS

sleep_time = DT_TARGET - (time.perf_counter() - now)
if sleep_time > 0:
    time.sleep(sleep_time)

with state_lock:

```

```

live_state["running"] = False
live_state["moving"] = False

# -----
# Save CSV
# -----
with open(OUTPUT_CSV, "w", newline="") as f:
    writer = csv.writer(f)
    writer.writerow([
        "time_s",
        "x_m",
        "y_m",
        "axis_heading_deg",
        "original_yaw_deg",
        "relative_heading_deg",
        "smoothed_heading_deg",
        "instant_quantized_heading_deg",
        "instant_quantized_heading_dir",
        "accepted_heading_deg",
        "accepted_heading_dir",
        "cal_sys",
        "cal_gyro",
        "cal_acc",
        "cal_mag",
    ])

    n = min(len(times), len(xs), len(ys), len(heading_t))
    for i in range(n):
        writer.writerow([
            times[i],
            xs[i],
            ys[i],
            raw_heading_deg_log[i],
            original_yaw_deg_log[i],
            rel_heading_deg_log[i],
            smooth_heading_deg_log[i],
            quant_heading_deg_log[i],
            quant_heading_dir_log[i],
            accepted_heading_deg_log[i],
            accepted_heading_dir_log[i],
            cal_sys_log[i],
            cal_gyro_log[i],
            cal_acc_log[i],
            cal_mag_log[i],
        ])

# -----
# XY trajectory plot
# -----
plt.figure(figsize=(7, 7))
plt.plot(xs, ys, linewidth=2, label="trajectory")

if xs and ys:
    plt.scatter([xs[0]], [ys[0]], s=70, label="start")
    plt.scatter([xs[-1]], [ys[-1]], s=70, label="end")

if step_events:
    step_xs = [e["x"] for e in step_events]
    step_ys = [e["y"] for e in step_events]
    plt.scatter(step_xs, step_ys, s=25, label="steps")

    if LABEL_STEP_TIMES:
        for e in step_events:
            label = f'{e["t"]:.1f}s {e["heading_used_dir"]}'
            plt.text(e["x"], e["y"], label, fontsize=7)

plt.title(f"BN0055 PDR XY Trajectory, Vertical IMU, Cardinal Locked, {BNO_MODE_NAME}")
plt.xlabel("X position (m), East/West relative")
plt.ylabel("Y position (m), North/South relative")
plt.grid(True)
plt.legend()
plt.savefig(OUTPUT_PNG_XY, dpi=150, bbox_inches="tight")
plt.close()

# -----
# Acceleration vs time plot
# -----
fig, axs = plt.subplots(3, 1, figsize=(10, 8), sharex=True)

axs[0].plot(acc_t, acc_x)
axs[0].set_ylabel("ax (m/s^2)")
axs[0].grid(True)
axs[0].set_title(f"BN0055 Linear Acceleration vs Time, {BNO_MODE_NAME}")

axs[1].plot(acc_t, acc_y)
axs[1].set_ylabel("ay (m/s^2)")
axs[1].grid(True)

axs[2].plot(acc_t, acc_z)
axs[2].set_ylabel("az (m/s^2)")
axs[2].set_xlabel("Time (s)")
axs[2].grid(True)

plt.tight_layout()

```

```

plt.savefig(OUTPUT_PNG_ACC, dpi=150, bbox_inches="tight")
plt.close()

# -----
# Step-signal plot with detected steps
# -----
plt.figure(figsize=(10, 5))
plt.plot(sig_t, sig_filt_log, label="filtered |a|")
plt.plot(sig_t, baseline_log, label="baseline")
plt.plot(sig_t, step_signal_log, label="step_signal")

if step_events:
    step_ts = [e["t"] for e in step_events]
    step_vals = []

    for ts in step_ts:
        idx = min(range(len(sig_t)), key=lambda i: abs(sig_t[i] - ts))
        step_vals.append(step_signal_log[idx])

    plt.scatter(step_ts, step_vals, s=30, label="detected steps")

    if LABEL_STEP_TIMES:
        for e, sv in zip(step_events, step_vals):
            plt.text(e["t"], sv, f'{e["t"]:.1f}s {e["heading_used_dir"]}', fontsize=7)

plt.axhline(STEP_EVENT_START, linestyle="--", linewidth=1, label="step event start")
plt.axhline(MIN_STEP_PEAK, linestyle=":", linewidth=1, label="min step peak")
plt.title(f"BN055 Step Signal with Detected Steps, {BNO_MODE_NAME}")
plt.xlabel("Time (s)")
plt.ylabel("Signal")
plt.grid(True)
plt.legend()
plt.savefig(OUTPUT_PNG_SIGNAL, dpi=150, bbox_inches="tight")
plt.close()

# -----
# Heading debug plot
# -----
plt.figure(figsize=(11, 6))

plt.plot(heading_t, raw_heading_deg_log, label=f"axis heading from {BODY_FORWARD_AXIS}")
plt.plot(heading_t, original_yaw_deg_log, label="original yaw formula, reference only", alpha=0.7)
plt.plot(heading_t, rel_heading_deg_log, label="relative heading, start = north")
plt.plot(heading_t, smooth_heading_deg_log, label="smoothed heading before quantization")
plt.step(heading_t, quant_heading_deg_log, where="post", label="instant quantized heading")
plt.step(heading_t, accepted_heading_deg_log, where="post", label="accepted locked direction")

if step_events:
    step_ts = [e["t"] for e in step_events]
    step_headings = [e["heading_deg"] for e in step_events]
    step_instant = [e["instant_heading_deg"] for e in step_events]
    step_used = [e["heading_used_deg"] for e in step_events]

    plt.scatter(step_ts, step_headings, s=30, label="continuous heading at steps")
    plt.scatter(step_ts, step_instant, s=40, marker="x", label="instant quantized at steps")
    plt.scatter(step_ts, step_used, s=45, marker="s", label="accepted locked at steps")

plt.axhline(0, linestyle="--", linewidth=1, label="north reference")
plt.axhline(90, linestyle=":", linewidth=1, label="+90 deg, East")
plt.axhline(-90, linestyle=":", linewidth=1, label="-90 deg, West")
plt.axhline(180, linestyle=":", linewidth=1, label="180 deg, South")
plt.axhline(-180, linestyle=":", linewidth=1)

plt.title(f"BN055 Heading Debug Plot, Vertical IMU, Axis={BODY_FORWARD_AXIS}, {BNO_MODE_NAME}")
plt.xlabel("Time (s)")
plt.ylabel("Heading (degrees)")
plt.grid(True)
plt.legend()
plt.savefig(OUTPUT_PNG_HEADING, dpi=150, bbox_inches="tight")
plt.close()

print(f"\nSaved XY plot to: {OUTPUT_PNG_XY}")
print(f"Saved accel plot to: {OUTPUT_PNG_ACC}")
print(f"Saved step signal to: {OUTPUT_PNG_SIGNAL}")
print(f"Saved heading plot to: {OUTPUT_PNG_HEADING}")
print(f"Saved CSV to: {OUTPUT_CSV}")
print(f"Detected steps: {len(step_events)}")
print(f"Mode used: {BNO_MODE_NAME}")
print(f"Body forward axis used: {BODY_FORWARD_AXIS}")
print(f"Heading offset used: {HEADING_OFFSET_DEG:.1f} deg")
print(f"Flipped heading sign: {USE_FLIPPED_HEADING_SIGN}")
print(f"Cardinal heading used: {USE_CARDINAL_HEADING}")
print(f"Flip X update: {FLIP_X_UPDATE}")
print(f"Direction switch to new: {DIRECTION_SWITCH_TO_NEW_DEG:.1f} deg")
print(f"Direction leave current: {DIRECTION_LEAVE_CURRENT_DEG:.1f} deg")
print(f"Direction confirm steps: {DIRECTION_CONFIRM_STEPS}")

print("\nFinal live state being served to Pi 4:")
with state_lock:
    print(json.dumps(live_state, indent=2))

print("\nThe Wi-Fi server is still alive until you press Ctrl+C.")
print(f"Pi 4 can still read: http://<PI_ZERO_IP>:{SERVER_PORT}/position")
try:

```

```
while True:
    time.sleep(1.0)
except KeyboardInterrupt:
    print("\nExiting Pi Zero PDR Wi-Fi server.")
```

5. motor_interface.py

```
#!/usr/bin/env python3
"""
motor_interface.py

Motor interface for maze_bot.

Important locked-PWM behavior:
- Subscribes to /motor_pwm_lock as Float32MultiArray:
  [left_pwm, right_pwm, duration_sec]
- When locked PWM is active:
  /cmd_vel is ignored completely.
  The locked left/right command is converted once into final motor outputs.
  The same final left/right outputs are applied repeatedly until duration ends.
- Locked PWM uses ratio-preserving scaling:
  motor scale factors are applied,
  then any min/max correction is applied to both sides together
  so the relative left/right steering ratio is not distorted mid-step.

This is intended to work with reactive_nav.py, where each autonomous step:
  calculate target once -> publish fixed PWM pair -> move fixed duration
"""

import math
import os
import time
from typing import Optional, Tuple

os.environ["GPIOZERO_PIN_FACTORY"] = "lgpio"

import rclpy
from rclpy.node import Node

from geometry_msgs.msg import Twist
from std_msgs.msg import Float32MultiArray

from gpiozero import PWMOutputDevice, DigitalOutputDevice

class MotorInterface(Node):
    def __init__(self) -> None:
        super().__init__("motor_interface")

        # ----- Parameters -----
        self.declare_parameter("cmd_vel_topic", "/cmd_vel")
        self.declare_parameter("motor_pwm_lock_topic", "/motor_pwm_lock")

        self.declare_parameter("min_pwm", 0.50)
        self.declare_parameter("max_pwm", 1.0)

        self.declare_parameter("linear_scale", 1.0)
        self.declare_parameter("angular_scale", 1.0)

        # Forward/backward were correct, but left/right turns needed this sign.
        self.declare_parameter("angular_sign", -1.0)

        # Hardware calibration scale.
        # These are applied before final output.
        self.declare_parameter("left_motor_scale", .895)
        self.declare_parameter("right_motor_scale", 0.92)

        self.declare_parameter("cmd_timeout_sec", 0.5)
        self.declare_parameter("control_rate_hz", 50.0)

        # GPIO pins
        self.declare_parameter("stby_pin", 17)

        self.declare_parameter("pwma_pin", 13)
        self.declare_parameter("ain1_pin", 15)
        self.declare_parameter("ain2_pin", 14)

        self.declare_parameter("pwmb_pin", 12)
        self.declare_parameter("bin1_pin", 21)
        self.declare_parameter("bin2_pin", 19)

        self.cmd_vel_topic = str(self.get_parameter("cmd_vel_topic").value)
        self.motor_pwm_lock_topic = str(self.get_parameter("motor_pwm_lock_topic").value)

        self.min_pwm = float(self.get_parameter("min_pwm").value)
        self.max_pwm = float(self.get_parameter("max_pwm").value)

        self.linear_scale = float(self.get_parameter("linear_scale").value)
        self.angular_scale = float(self.get_parameter("angular_scale").value)
        self.angular_sign = float(self.get_parameter("angular_sign").value)

        self.left_motor_scale = float(self.get_parameter("left_motor_scale").value)
        self.right_motor_scale = float(self.get_parameter("right_motor_scale").value)

        self.cmd_timeout_sec = float(self.get_parameter("cmd_timeout_sec").value)
        control_rate_hz = float(self.get_parameter("control_rate_hz").value)

        stby_pin = int(self.get_parameter("stby_pin").value)

        pwma_pin = int(self.get_parameter("pwma_pin").value)
        ain1_pin = int(self.get_parameter("ain1_pin").value)
```

```

ain2_pin = int(self.get_parameter("ain2_pin").value)

pwmb_pin = int(self.get_parameter("pwmb_pin").value)
bin1_pin = int(self.get_parameter("bin1_pin").value)
bin2_pin = int(self.get_parameter("bin2_pin").value)

# ----- GPIO setup -----
self.stby = DigitalOutputDevice(stby_pin)

self.left_pwm = PWMOutputDevice(pwma_pin, frequency=1000)
self.left_in1 = DigitalOutputDevice(ain1_pin)
self.left_in2 = DigitalOutputDevice(ain2_pin)

self.right_pwm = PWMOutputDevice(pwmb_pin, frequency=1000)
self.right_in1 = DigitalOutputDevice(bin1_pin)
self.right_in2 = DigitalOutputDevice(bin2_pin)

self.stby.on()

# ----- ROS I/O -----
self.cmd_sub = self.create_subscription(
    Twist,
    self.cmd_vel_topic,
    self.cmd_vel_callback,
    10,
)

self.lock_sub = self.create_subscription(
    Float32MultiArray,
    self.motor_pwm_lock_topic,
    self.locked_pwm_callback,
    10,
)

self.timer = self.create_timer(
    1.0 / max(control_rate_hz, 1.0),
    self.control_loop,
)

# ----- State -----
self.last_cmd_time = 0.0
self.last_cmd: Optional[Twist] = None

self.locked_pwm_active = False
self.locked_pwm_end_time = 0.0

# Raw command received from /motor_pwm_lock.
self.locked_raw_left_pwm = 0.0
self.locked_raw_right_pwm = 0.0

# Final motor outputs calculated once at lock start.
# These are what get repeatedly applied during the whole locked step.
self.locked_output_left_pwm = 0.0
self.locked_output_right_pwm = 0.0

self.last_locked_log_time = 0.0

self.get_logger().info("Motor interface started: ratio-preserving locked PWM version.")
self.get_logger().info(f"Listening on {self.cmd_vel_topic} and {self.motor_pwm_lock_topic}")
self.get_logger().info(
    f"Pins: STBY={stby_pin}, "
    f"L(PWM={pwma_pin}, IN1={ain1_pin}, IN2={ain2_pin}), "
    f"R(PWM={pwmb_pin}, IN1={bin1_pin}, IN2={bin2_pin})"
)
self.get_logger().info(
    f"min_pwm={self.min_pwm:.2f}, max_pwm={self.max_pwm:.2f}, "
    f"left_motor_scale={self.left_motor_scale:.3f}, "
    f"right_motor_scale={self.right_motor_scale:.3f}"
)
self.get_logger().info(
    "While locked_pwm_active=True, /cmd_vel is ignored and the same "
    "final left/right PWM outputs are held until duration expires."
)

# ----- Callbacks -----

def cmd_vel_callback(self, msg: Twist) -> None:
    if self.locked_pwm_active:
        # Autonomous fixed movement step owns the motors.
        # Do not let /cmd_vel modify motor outputs mid-step.
        return

    self.last_cmd = msg
    self.last_cmd_time = time.monotonic()

def locked_pwm_callback(self, msg: Float32MultiArray) -> None:
    if len(msg.data) < 3:
        self.get_logger().warn(
            "Ignoring /motor_pwm_lock message with fewer than 3 values."
        )
        return

    raw_left = float(msg.data[0])
    raw_right = float(msg.data[1])
    duration = float(msg.data[2])

    if duration <= 0.0 or (abs(raw_left) < 1e-6 and abs(raw_right) < 1e-6):

```



```

        self.locked_pwm_active = False
        self.locked_raw_left_pwm = 0.0
        self.locked_raw_right_pwm = 0.0
        self.locked_output_left_pwm = 0.0
        self.locked_output_right_pwm = 0.0
        self.locked_pwm_end_time = 0.0
        self.stop_motors()
        self.get_logger().info("Locked PWM cancelled/stopped.")
        return

# Clamp raw command to allowed range, but do not min-clamp here.
# The locked final outputs are computed using ratio-preserving logic.
raw_left = self.clamp_max_only(raw_left)
raw_right = self.clamp_max_only(raw_right)

final_left, final_right = self.prepare_locked_outputs_preserve_ratio(
    raw_left,
    raw_right,
)

self.locked_raw_left_pwm = raw_left
self.locked_raw_right_pwm = raw_right

self.locked_output_left_pwm = final_left
self.locked_output_right_pwm = final_right

self.locked_pwm_end_time = time.monotonic() + duration
self.locked_pwm_active = True
self.last_locked_log_time = 0.0

self.get_logger().info(
    "Locked PWM active: "
    f"RAW L={raw_left:.3f}, R={raw_right:.3f}, "
    f"raw_ratio={self.safe_ratio(raw_left, raw_right):.3f}; "
    f"FINAL L={final_left:.3f}, R={final_right:.3f}, "
    f"final_ratio={self.safe_ratio(final_left, final_right):.3f}; "
    f"duration={duration:.2f}s"
)

# ----- Control -----

def control_loop(self) -> None:
    now = time.monotonic()

    if self.locked_pwm_active:
        if now >= self.locked_pwm_end_time:
            self.locked_pwm_active = False

            finished_left = self.locked_output_left_pwm
            finished_right = self.locked_output_right_pwm

            self.locked_raw_left_pwm = 0.0
            self.locked_raw_right_pwm = 0.0
            self.locked_output_left_pwm = 0.0
            self.locked_output_right_pwm = 0.0
            self.locked_pwm_end_time = 0.0

            self.stop_motors()

            self.get_logger().info(
                f"Locked PWM finished. Motors stopped. "
                f"Last held outputs: L={finished_left:.3f}, R={finished_right:.3f}"
            )
        else:
            # This is the actual lock:
            # repeatedly apply the same final motor outputs.
            self.set_motor_outputs_direct(
                self.locked_output_left_pwm,
                self.locked_output_right_pwm,
            )

            if now - self.last_locked_log_time > 0.5:
                remaining = max(0.0, self.locked_pwm_end_time - now)
                self.get_logger().debug(
                    f"Holding locked PWM: "
                    f"L={self.locked_output_left_pwm:.3f}, "
                    f"R={self.locked_output_right_pwm:.3f}, "
                    f"remaining={remaining:.2f}s"
                )
                self.last_locked_log_time = now

    return

# Manual /cmd_vel path.
if self.last_cmd is None or (now - self.last_cmd_time) > self.cmd_timeout_sec:
    self.stop_motors()
    return

left, right = self.twist_to_pwms(self.last_cmd)
self.set_motor_outputs_direct(left, right)

def twist_to_pwms(self, twist: Twist) -> Tuple[float, float]:
    linear = float(twist.linear.x) * self.linear_scale
    angular = float(twist.angular.z) * self.angular_scale * self.angular_sign

    left = linear - angular
    right = linear + angular

```

```

# For manual driving, independent min clamp is okay.
left = self.clamp_with_min(left)
right = self.clamp_with_min(right)

# Apply hardware motor scale for manual commands too.
left *= self.left_motor_scale
right *= self.right_motor_scale

left = self.clamp_with_min(left)
right = self.clamp_with_min(right)

return left, right

# ----- Locked PWM ratio-preserving helpers -----

def prepare_locked_outputs_preserve_ratio(
    self,
    raw_left: float,
    raw_right: float,
) -> Tuple[float, float]:
    """
    Convert raw locked command into final motor outputs.

    This is intentionally different from manual cmd_vel handling.

    For locked PWM:
    - Apply left/right motor calibration scales.
    - Do NOT independently clamp each side to min_pwm.
    - If one side is under min_pwm, scale BOTH sides by the same factor.
    - If one side exceeds max_pwm, scale BOTH sides down by the same factor.
    - The final left/right values are stored and reused for the entire step.
    """

    # Apply motor calibration scales.
    left = raw_left * self.left_motor_scale
    right = raw_right * self.right_motor_scale

    # If both are basically zero, stop.
    if abs(left) < 1e-6 and abs(right) < 1e-6:
        return 0.0, 0.0

    # First clamp absurd raw values by common scaling if needed.
    max_abs = max(abs(left), abs(right), 1e-6)

    if max_abs > self.max_pwm:
        scale_down = self.max_pwm / max_abs
        left *= scale_down
        right *= scale_down

    # Enforce min PWM using common scaling, preserving ratio.
    nonzero_values = [abs(v) for v in (left, right) if abs(v) > 1e-6]

    if nonzero_values:
        min_abs_nonzero = min(nonzero_values)

        if min_abs_nonzero < self.min_pwm:
            scale_up = self.min_pwm / max(min_abs_nonzero, 1e-6)
            left *= scale_up
            right *= scale_up

    # If min scaling pushed one side too high, scale both down together.
    max_abs = max(abs(left), abs(right), 1e-6)

    if max_abs > self.max_pwm:
        scale_down = self.max_pwm / max_abs
        left *= scale_down
        right *= scale_down

    # Final numerical clamp only.
    left = self.clamp_max_only(left)
    right = self.clamp_max_only(right)

    return left, right

def clamp_max_only(self, value: float) -> float:
    return max(-self.max_pwm, min(self.max_pwm, value))

def clamp_with_min(self, value: float) -> float:
    if abs(value) < 1e-6:
        return 0.0

    value = self.clamp_max_only(value)

    if abs(value) < self.min_pwm:
        value = math.copysign(self.min_pwm, value)

    return value

def safe_ratio(self, left: float, right: float) -> float:
    if abs(right) < 1e-6:
        return 999.0
    return left / right

# ----- Motor GPIO helpers -----

def set_motor_outputs_direct(self, left: float, right: float) -> None:
    """

```

Apply final motor outputs directly.

This function does NOT do ratio-changing min PWM logic.
It assumes locked PWM commands have already been prepared.
"""

```
left = self.clamp_max_only(left)
right = self.clamp_max_only(right)
```

```
self.set_one_motor(left, self.left_pwm, self.left_in1, self.left_in2)
self.set_one_motor(right, self.right_pwm, self.right_in1, self.right_in2)
```

```
def set_one_motor(
    self,
    pwm_value: float,
    pwm_dev: PWMOutputDevice,
    in1: DigitalOutputDevice,
    in2: DigitalOutputDevice,
) -> None:
    pwm_value = self.clamp_max_only(pwm_value)
```

```
    if abs(pwm_value) < 1e-6:
        in1.off()
        in2.off()
        pwm_dev.value = 0.0
        return
```

```
    if pwm_value > 0.0:
        in1.on()
        in2.off()
    else:
        in1.off()
        in2.on()
```

```
    pwm_dev.value = min(abs(pwm_value), 1.0)
```

```
def stop_motors(self) -> None:
    self.left_pwm.value = 0.0
    self.right_pwm.value = 0.0
```

```
    self.left_in1.off()
    self.left_in2.off()
```

```
    self.right_in1.off()
    self.right_in2.off()
```

```
def destroy_node(self) -> bool:
    self.stop_motors()
    self.stby.off()
    return super().destroy_node()
```

```
def main(args=None) -> None:
    rclpy.init(args=args)
    node = MotorInterface()
```

```
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        node.stop_motors()
        node.destroy_node()
        rclpy.shutdown()
```

```
if __name__ == "__main__":
    main()
```

6. reactive_nav.py

```
#!/usr/bin/env python3
"""
reactive_nav.py

Reactive LiDAR navigation for obstacle-course behavior.

Main behavior:
- Uses /scan from SLLIDAR.
- Continuously computes live RViz markers from the newest scan.
- start / step_run / run / auto = continuous stepping.
- step = one locked movement step.
- During each movement step:
  1. Calculate the best open ray / gap once.
  2. Convert that target angle into fixed left/right PWM.
  3. Publish /motor_pwm_lock:
      Float32MultiArray [left_pwm, right_pwm, duration_sec]
  4. Do NOT change motor PWM during the step.
  5. Recalculate only after the step finishes.
- If an obstacle is too close directly ahead:
  1. Compare long rays on left vs. right.
  2. Lock a turn-in-place toward the more open side.
  3. After the turn, recalculate using the newest scan.

Updated:
- Navigation view cone is now -60 deg to +60 deg, 120 deg total.
- Candidate rays require at least min_cluster_points nearby scan points.
- Default min_cluster_points is now 15.

Live RViz marker topics:
- /nav_filtered_profile
- /nav_target_cluster
- /nav_top_candidates
- /nav_target_marker
- /nav_locked_target_marker
- /nav_fov_bounds_marker
"""

import math
import time
from typing import Any, Dict, List, Optional, Tuple

import rclpy
from rclpy.node import Node
from rclpy.qos import (
    QoSDurationPolicy,
    QoSHistoryPolicy,
    QoSProfile,
    QoSReliabilityPolicy,
)

from geometry_msgs.msg import Point, Twist
from sensor_msgs.msg import LaserScan
from std_msgs.msg import Float32MultiArray, String
from visualization_msgs.msg import Marker

RayPoint = Tuple[float, float, float, float, float]
# angle_rad_robot_frame, range_m, x_m, y_m, front_bumper_clearance_m

class ReactiveNav(Node):
    def __init__(self) -> None:
        super().__init__("reactive_nav")

        # ----- ROS parameters -----
        self.declare_parameter("scan_topic", "/scan")
        self.declare_parameter("cmd_vel_topic", "/cmd_vel")
        self.declare_parameter("nav_mode_topic", "/nav_mode")

        # LiDAR geometry/orientation
        self.declare_parameter("lidar_scan_offset_deg", 0.0)
        self.declare_parameter("lidar_to_front_offset_m", 0.203)

        # Direction signs
        self.declare_parameter("steering_sign", 1.0)
        self.declare_parameter("linear_sign", 1.0)

        # Navigation FOV: 120 degrees total
        # -60 degrees to +60 degrees.
        self.declare_parameter("fov_left_deg", 60.0)
        self.declare_parameter("fov_right_deg", -60.0)

        # Scan filtering
        self.declare_parameter("min_range_m", 0.12)
        self.declare_parameter("max_range_m", 5.0)

        # Candidate selectivity
        # A candidate ray must have at least this many nearby points in its
        # local angular window to be considered stable enough.
        self.declare_parameter("min_cluster_points", 15)

        # Step timing
        self.declare_parameter("drive_step_duration_sec", 2.75)
        self.declare_parameter("avoid_turn_duration_sec", 0.85)
```

```

# PWM behavior
self.declare_parameter("min_pwm", 0.50)
self.declare_parameter("max_forward_pwm", 0.85)
self.declare_parameter("calibration_pwm", 0.80)
self.declare_parameter("turn_pwm_gain", 0.42)

# Avoidance behavior
self.declare_parameter("close_obstacle_clearance_m", 0.25)
self.declare_parameter("front_clearance_window_deg", 18.0)
self.declare_parameter("side_ray_min_deg", 25.0)
self.declare_parameter("side_ray_max_deg", 60.0)
self.declare_parameter("avoid_turn_pwm", 0.62)

# Open-ray scoring
self.declare_parameter("candidate_downsample", 2)
self.declare_parameter("local_window_deg", 12.0)
self.declare_parameter("long_ray_threshold_m", 1.00)
self.declare_parameter("target_marker_range_m", 1.8)

# Update loop
self.declare_parameter("control_rate_hz", 10.0)

self.scan_topic = str(self.get_parameter("scan_topic").value)
self.cmd_vel_topic = str(self.get_parameter("cmd_vel_topic").value)
self.nav_mode_topic = str(self.get_parameter("nav_mode_topic").value)

self.lidar_scan_offset_rad = math.radians(
    float(self.get_parameter("lidar_scan_offset_deg").value)
)
self.lidar_to_front_offset_m = float(
    self.get_parameter("lidar_to_front_offset_m").value
)

self.steering_sign = float(self.get_parameter("steering_sign").value)
self.linear_sign = float(self.get_parameter("linear_sign").value)

self.fov_left_rad = math.radians(float(self.get_parameter("fov_left_deg").value))
self.fov_right_rad = math.radians(float(self.get_parameter("fov_right_deg").value))
self.fov_min_rad = min(self.fov_left_rad, self.fov_right_rad)
self.fov_max_rad = max(self.fov_left_rad, self.fov_right_rad)

self.min_range_m = float(self.get_parameter("min_range_m").value)
self.max_range_m = float(self.get_parameter("max_range_m").value)

self.min_cluster_points = int(self.get_parameter("min_cluster_points").value)

self.drive_step_duration_sec = float(
    self.get_parameter("drive_step_duration_sec").value
)
self.avoid_turn_duration_sec = float(
    self.get_parameter("avoid_turn_duration_sec").value
)

self.min_pwm = float(self.get_parameter("min_pwm").value)
self.max_forward_pwm = float(self.get_parameter("max_forward_pwm").value)
self.calibration_pwm = float(self.get_parameter("calibration_pwm").value)
self.turn_pwm_gain = float(self.get_parameter("turn_pwm_gain").value)

self.close_obstacle_clearance_m = float(
    self.get_parameter("close_obstacle_clearance_m").value
)
self.front_clearance_window_rad = math.radians(
    float(self.get_parameter("front_clearance_window_deg").value)
)
self.side_ray_min_rad = math.radians(
    float(self.get_parameter("side_ray_min_deg").value)
)
self.side_ray_max_rad = math.radians(
    float(self.get_parameter("side_ray_max_deg").value)
)
self.avoid_turn_pwm = float(self.get_parameter("avoid_turn_pwm").value)

self.candidate_downsample = max(
    1,
    int(self.get_parameter("candidate_downsample").value),
)
self.local_window_rad = math.radians(
    float(self.get_parameter("local_window_deg").value)
)
self.long_ray_threshold_m = float(
    self.get_parameter("long_ray_threshold_m").value
)
self.target_marker_range_m = float(
    self.get_parameter("target_marker_range_m").value
)

control_rate_hz = float(self.get_parameter("control_rate_hz").value)
self.control_period_sec = 1.0 / max(control_rate_hz, 1.0)

# Marker QoS
self.marker_qos = QoSProfile(
    history=QoSHistoryPolicy.KEEP_LAST,
    depth=10,
    reliability=QoSReliabilityPolicy.RELIABLE,
    durability=QoSDurabilityPolicy.TRANSIENT_LOCAL,
)
# ----- ROS I/O -----

```

```

self.scan_sub = self.create_subscription(
    LaserScan,
    self.scan_topic,
    self.scan_callback,
    10,
)

self.nav_mode_sub = self.create_subscription(
    String,
    self.nav_mode_topic,
    self.nav_mode_callback,
    10,
)

self.cmd_vel_pub = self.create_publisher(Twist, self.cmd_vel_topic, 10)

self.motor_pwm_lock_pub = self.create_publisher(
    Float32MultiArray,
    "/motor_pwm_lock",
    10,
)

self.filtered_profile_pub = self.create_publisher(
    Marker,
    "/nav_filtered_profile",
    self.marker_qos,
)

self.target_cluster_pub = self.create_publisher(
    Marker,
    "/nav_target_cluster",
    self.marker_qos,
)

self.top_candidates_pub = self.create_publisher(
    Marker,
    "/nav_top_candidates",
    self.marker_qos,
)

self.target_marker_pub = self.create_publisher(
    Marker,
    "/nav_target_marker",
    self.marker_qos,
)

self.locked_target_marker_pub = self.create_publisher(
    Marker,
    "/nav_locked_target_marker",
    self.marker_qos,
)

self.fov_bounds_pub = self.create_publisher(
    Marker,
    "/nav_fov_bounds_marker",
    self.marker_qos,
)

self.timer = self.create_timer(self.control_period_sec, self.control_loop)

# ----- State -----
self.latest_scan: Optional[LaserScan] = None

self.state = "STOPPED"
self.continuous_step_run = False

self.move_end_time = 0.0

self.locked_target_xy: Optional[Tuple[float, float]] = None
self.locked_left_pwm = 0.0
self.locked_right_pwm = 0.0
self.locked_action = "NONE"

self.last_no_scan_log_time = 0.0
self.last_hold_log_time = 0.0

self.get_logger().info("Reactive nav started: selective open-ray obstacle-course version.")
self.get_logger().info(f"Subscribing to scan topic: {self.scan_topic}")
self.get_logger().info(
    "FOV: "
    f"{math.degrees(self.fov_min_rad):.1f} deg to "
    f"{math.degrees(self.fov_max_rad):.1f} deg "
    f"({math.degrees(self.fov_max_rad - self.fov_min_rad):.1f} deg total)"
)
self.get_logger().info(
    f"Minimum nearby points for candidate ray: {self.min_cluster_points}"
)
self.get_logger().info(
    f"Close obstacle turn threshold: {self.close_obstacle_clearance_m:.2f} m"
)
self.get_logger().info(
    "Behavior: close obstacle -> locked turn toward longer side ray; "
    "otherwise locked curved step toward best open ray."
)

# ----- Callbacks -----

def scan_callback(self, msg: LaserScan) -> None:
    self.latest_scan = msg

def nav_mode_callback(self, msg: String) -> None:
    cmd = msg.data.strip().lower()

```

```

if cmd in ("start", "step_run", "run", "auto"):
    self.continuous_step_run = True

    if self.state != "MOVING":
        self.state = "CALCULATE"

    self.get_logger().info("Navigation command: START continuous stepping")

elif cmd == "step":
    self.continuous_step_run = False

    if self.state != "MOVING":
        self.state = "CALCULATE"

    self.get_logger().info("Navigation command: STEP once")

elif cmd in ("stop", "halt", "off"):
    self.continuous_step_run = False
    self.state = "STOPPED"

    self.locked_target_xy = None
    self.locked_left_pwm = 0.0
    self.locked_right_pwm = 0.0
    self.locked_action = "NONE"

    self.publish_stop()
    self.publish_zero_pwm_lock()
    self.delete_locked_target_marker()

    self.get_logger().info("Navigation command: STOP")

else:
    self.get_logger().warn(f"Unknown /nav_mode command: {cmd}")

# ----- Main loop -----

def control_loop(self) -> None:
    now = time.monotonic()

    if self.latest_scan is None:
        self.publish_fov_bounds()

        if now - self.last_no_scan_log_time > 2.0:
            self.get_logger().warn("No /scan received yet.")
            self.last_no_scan_log_time = now

        if self.state != "STOPPED":
            self.publish_stop()

    return

# Always recompute marker/candidate data from the newest scan.
# This gives real-time RViz updates even while moving.
processed = self.process_scan(self.latest_scan)
self.publish_live_markers(processed)

if self.state == "STOPPED":
    self.publish_stop()
    return

if self.state == "MOVING":
    # Critical: do not recompute or republish motor PWM while moving.
    # The motor interface holds the locked PWM until duration ends.
    if now >= self.move_end_time:
        self.publish_stop()

        finished_action = self.locked_action
        finished_left = self.locked_left_pwm
        finished_right = self.locked_right_pwm

        self.locked_target_xy = None
        self.locked_left_pwm = 0.0
        self.locked_right_pwm = 0.0
        self.locked_action = "NONE"
        self.delete_locked_target_marker()

        if self.continuous_step_run:
            self.state = "CALCULATE"
            self.get_logger().info(
                f"Continuous mode: {finished_action} step finished "
                f"(L={finished_left:.2f}, R={finished_right:.2f}); "
                "recalculating."
            )
        else:
            self.state = "STOPPED"
            self.get_logger().info(
                f"Single {finished_action} step finished "
                f"(L={finished_left:.2f}, R={finished_right:.2f}); "
                "returning to STOPPED."
            )

    return

if self.state == "CALCULATE":
    self.calculate_and_start_locked_step(processed)
    return
self.get_logger().warn(f"Unknown state {self.state}; stopping.")

```

```

self.state = "STOPPED"
self.continuous_step_run = False
self.publish_stop()
self.publish_zero_pwm_lock()

# ----- Scan processing -----

def process_scan(self, scan: LaserScan) -> Dict[str, Any]:
    rays = self.extract_open_rays(scan)
    front_clearance = self.compute_front_clearance(rays)
    candidates = self.score_open_rays(rays)
    side_info = self.compute_side_opening(rays)

    return {
        "rays": rays,
        "front_clearance": front_clearance,
        "candidates": candidates,
        "side_info": side_info,
    }

def extract_open_rays(self, scan: LaserScan) -> List[RayPoint]:
    rays: List[RayPoint] = []

    angle = scan.angle_min

    for raw_r in scan.ranges:
        if math.isinf(raw_r):
            r = self.max_range_m
        elif math.isfinite(raw_r):
            r = float(raw_r)
        else:
            angle += scan.angle_increment
            continue

        if r < self.min_range_m:
            angle += scan.angle_increment
            continue

        r = min(r, self.max_range_m)

        a = angle + self.lidar_scan_offset_rad
        a = math.atan2(math.sin(a), math.cos(a))

        if self.fov_min_rad <= a <= self.fov_max_rad:
            x = r * math.cos(a)
            y = r * math.sin(a)

            if x > 0.0:
                front_bumper_clearance = max(0.0, x - self.lidar_to_front_offset_m)
                rays.append((a, r, x, y, front_bumper_clearance))

        angle += scan.angle_increment

    rays.sort(key=lambda p: p[0])
    return rays

def compute_front_clearance(self, rays: List[RayPoint]) -> float:
    front_rays = [p for p in rays if abs(p[0]) <= self.front_clearance_window_rad]

    if not front_rays:
        return self.max_range_m

    return min(p[4] for p in front_rays)

def compute_side_opening(self, rays: List[RayPoint]) -> Dict[str, float]:
    left_rays = [
        p for p in rays
        if self.side_ray_min_rad <= p[0] <= self.side_ray_max_rad
    ]
    right_rays = [
        p for p in rays
        if -self.side_ray_max_rad <= p[0] <= -self.side_ray_min_rad
    ]

    left_score = self.side_open_score(left_rays)
    right_score = self.side_open_score(right_rays)

    return {
        "left_score": left_score,
        "right_score": right_score,
    }

def side_open_score(self, rays: List[RayPoint]) -> float:
    if not rays:
        return 0.0

    ranges = [p[1] for p in rays]
    long_ranges = [r for r in ranges if r >= self.long_ray_threshold_m]

    max_r = max(ranges)

    if long_ranges:
        avg_long = sum(long_ranges) / len(long_ranges)
    else:
        avg_long = sum(ranges) / len(ranges)

    return 0.65 * max_r + 0.35 * avg_long

```



```

def score_open_rays(self, rays: List[RayPoint]) -> List[dict]:
    candidates = []

    if not rays:
        return candidates

    for i, ray in enumerate(rays):
        if i % self.candidate_downsample != 0:
            continue

        angle, r, x, y, front_clearance = ray

        local_rays = [
            p for p in rays
            if abs(p[0] - angle) <= self.local_window_rad
        ]

        # This is the new selectivity requirement.
        # Do not consider isolated/noisy rays as candidates.
        if len(local_rays) < self.min_cluster_points:
            continue

        local_ranges = [p[1] for p in local_rays]
        local_mean_range = sum(local_ranges) / len(local_ranges)
        long_count = sum(1 for p in local_rays if p[1] >= self.long_ray_threshold_m)
        local_long_fraction = long_count / max(1, len(local_rays))

        range_score = min(r / self.max_range_m, 1.0)
        clearance_score = min(front_clearance / self.max_range_m, 1.0)

        # Prefer forward, but still allow side rays when they are much more open.
        center_score = 1.0 - min(abs(angle) / max(abs(self.fov_max_rad), 1e-6), 1.0)

        local_range_score = min(local_mean_range / self.max_range_m, 1.0)

        # Main goal: find a long, stable ray/gap.
        score = (
            3.0 * range_score
            + 2.0 * clearance_score
            + 1.2 * local_range_score
            + 1.0 * local_long_fraction
            + 0.8 * center_score
        )

        marker_dist = min(r, self.target_marker_range_m)
        target_x = marker_dist * math.cos(angle)
        target_y = marker_dist * math.sin(angle)

        candidates.append(
            {
                "score": score,
                "angle": angle,
                "range": r,
                "target_xy": (target_x, target_y),
                "raw_xy": (x, y),
                "front_bumper_clearance": front_clearance,
                "local_mean_range": local_mean_range,
                "local_long_fraction": local_long_fraction,
                "local_point_count": len(local_rays),
            }
        )

    candidates.sort(key=lambda item: item["score"], reverse=True)
    return candidates

# ----- Step action selection -----

def calculate_and_start_locked_step(self, processed: Dict[str, Any]) -> None:
    now = time.monotonic()

    rays: List[RayPoint] = processed["rays"]
    front_clearance: float = processed["front_clearance"]
    candidates: List[dict] = processed["candidates"]
    side_info: Dict[str, float] = processed["side_info"]

    if not rays:
        self.publish_stop()

    if now - self.last_hold_log_time > 1.0:
        self.get_logger().warn("No valid open rays. Holding position.")
        self.last_hold_log_time = now

    self.state = "CALCULATE" if self.continuous_step_run else "STOPPED"
    return

if front_clearance <= self.close_obstacle_clearance_m:
    action, left_pwm, right_pwm, duration, locked_xy = self.make_avoid_turn_command(
        side_info
    )
    self.start_locked_action(
        action=action,
        left_pwm=left_pwm,
        right_pwm=right_pwm,
        duration=duration,
        target_xy=locked_xy,
        extra_log=(
            f"front_clearance={front_clearance:.2f}, "

```

```

        f"left_open={side_info['left_score']:.2f}, "
        f"right_open={side_info['right_score']:.2f}"
    ),
)
return

if not candidates:
    self.publish_stop()

    if now - self.last_hold_log_time > 1.0:
        self.get_logger().warn(
            f"No open-ray candidates with at least {self.min_cluster_points} local points. "
            "Holding position."
        )
        self.last_hold_log_time = now

    self.state = "CALCULATE" if self.continuous_step_run else "STOPPED"
    return

best = candidates[0]
target_angle = float(best["angle"])
target_x, target_y = best["target_xy"]

left_pwm, right_pwm = self.target_angle_to_locked_pwm(target_angle)

self.start_locked_action(
    action="TRACK_RAY",
    left_pwm=left_pwm,
    right_pwm=right_pwm,
    duration=self.drive_step_duration_sec,
    target_xy=(target_x, target_y),
    extra_log=(
        f"angle={math.degrees(target_angle):.1f} deg, "
        f"ray_range={best['range']:.2f}, "
        f"front_clearance={front_clearance:.2f}, "
        f"local_points={best['local_point_count']}, "
        f"score={best['score']:.2f}"
    ),
)

def start_locked_action(
    self,
    action: str,
    left_pwm: float,
    right_pwm: float,
    duration: float,
    target_xy: Tuple[float, float],
    extra_log: str,
) -> None:
    self.locked_action = action
    self.locked_left_pwm = left_pwm
    self.locked_right_pwm = right_pwm
    self.locked_target_xy = target_xy

    self.publish_locked_target_marker(target_xy[0], target_xy[1])

    msg = Float32MultiArray()
    msg.data = [float(left_pwm), float(right_pwm), float(duration)]
    self.motor_pwm_lock_pub.publish(msg)

    self.move_end_time = time.monotonic() + duration
    self.state = "MOVING"

    mode_text = "continuous" if self.continuous_step_run else "single-step"

    self.get_logger().info(
        f"Locked {action} step started [{mode_text}]: "
        f"L={left_pwm:.2f}, R={right_pwm:.2f}, dur={duration:.2f}s, "
        f"{extra_log}"
    )

def make_avoid_turn_command(
    self,
    side_info: Dict[str, float],
) -> Tuple[str, float, float, float, Tuple[float, float]]:
    left_score = side_info["left_score"]
    right_score = side_info["right_score"]

    turn_left = left_score >= right_score

    p = max(self.min_pwm, min(self.max_forward_pwm, self.avoid_turn_pwm))

    if self.steering_sign < 0.0:
        turn_left = not turn_left

    if turn_left:
        left_pwm = -p
        right_pwm = p
        action = "AVOID_TURN_LEFT"
        target_angle = math.radians(60.0)
    else:
        left_pwm = p
        right_pwm = -p
        action = "AVOID_TURN_RIGHT"
        target_angle = math.radians(-60.0)

    marker_x = self.target_marker_range_m * math.cos(target_angle)

```

```

marker_y = self.target_marker_range_m * math.sin(target_angle)

return action, left_pwm, right_pwm, self.avoid_turn_duration_sec, (marker_x, marker_y)

def target_angle_to_locked_pwm(self, target_angle: float) -> Tuple[float, float]:
    """
    Convert target angle to a fixed curved PWM pair.

    This is intentionally calculated only at the beginning of the step.
    During MOVING, this function is not called again.
    """
    base_pwm = max(self.min_pwm, min(self.max_forward_pwm, self.calibration_pwm))

    angle_norm = target_angle / math.radians(60.0)
    angle_norm = max(-1.0, min(1.0, angle_norm))

    turn = self.steering_sign * self.turn_pwm_gain * angle_norm
    turn = max(-0.45, min(0.45, turn))

    raw_left = 1.0 - turn
    raw_right = 1.0 + turn

    max_raw = max(abs(raw_left), abs(raw_right), 1e-6)

    ratio_left = raw_left / max_raw
    ratio_right = raw_right / max_raw

    left = base_pwm * ratio_left * self.linear_sign
    right = base_pwm * ratio_right * self.linear_sign

    left, right = self.enforce_min_max_preserve_ratio(left, right)

    return left, right

def enforce_min_max_preserve_ratio(self, left: float, right: float) -> Tuple[float, float]:
    nonzero_values = [abs(v) for v in (left, right) if abs(v) > 1e-6]

    if nonzero_values:
        min_abs_nonzero = min(nonzero_values)

        if min_abs_nonzero < self.min_pwm:
            scale_up = self.min_pwm / max(min_abs_nonzero, 1e-6)
            left *= scale_up
            right *= scale_up

        max_abs = max(abs(left), abs(right), 1e-6)

        if max_abs > self.max_forward_pwm:
            scale_down = self.max_forward_pwm / max_abs
            left *= scale_down
            right *= scale_down

        left = max(-self.max_forward_pwm, min(self.max_forward_pwm, left))
        right = max(-self.max_forward_pwm, min(self.max_forward_pwm, right))

    return left, right

# ----- Command publishers -----

def publish_stop(self) -> None:
    self.cmd_vel_pub.publish(Twist())

def publish_zero_pwm_lock(self) -> None:
    msg = Float32MultiArray()
    msg.data = [0.0, 0.0, 0.0]
    self.motor_pwm_lock_pub.publish(msg)

# ----- Marker publishing -----

def publish_live_markers(self, processed: Dict[str, Any]) -> None:
    rays: List[RayPoint] = processed["rays"]
    candidates: List[dict] = processed["candidates"]

    self.publish_filtered_profile(rays)
    self.publish_target_clusters(rays)
    self.publish_top_candidates(candidates[:8])
    self.publish_fov_bounds()

    if candidates:
        best = candidates[0]
        x, y = best["target_xy"]
        self.publish_target_marker(x, y)
    else:
        self.delete_target_marker()

    if self.locked_target_xy is not None:
        self.publish_locked_target_marker(
            self.locked_target_xy[0],
            self.locked_target_xy[1],
        )
    else:
        self.delete_locked_target_marker()

def make_marker(self, ns: str, marker_id: int, marker_type: int) -> Marker:
    m = Marker()
    m.header.frame_id = "laser"
    m.header.stamp = self.get_clock().now().to_msg()

```

```

m.ns = ns
m.id = marker_id
m.type = marker_type
m.action = Marker.ADD
m.pose.orientation.w = 1.0
m.lifetime.sec = 0
m.lifetime.nanosec = 0
return m

def publish_filtered_profile(self, rays: List[RayPoint]) -> None:
    m = self.make_marker("nav_filtered_profile", 0, Marker.POINTS)
    m.scale.x = 0.035
    m.scale.y = 0.035
    m.color.a = 1.0
    m.color.r = 0.0
    m.color.g = 1.0
    m.color.b = 0.0

    for _, _, x, y, _ in rays:
        p = Point()
        p.x = float(x)
        p.y = float(y)
        p.z = 0.02
        m.points.append(p)

    self.filtered_profile_pub.publish(m)

def publish_target_clusters(self, rays: List[RayPoint]) -> None:
    """
    Blue points represent open-ray endpoints used for navigation.
    """
    m = self.make_marker("nav_target_cluster", 0, Marker.POINTS)
    m.scale.x = 0.055
    m.scale.y = 0.055
    m.color.a = 1.0
    m.color.r = 0.0
    m.color.g = 0.2
    m.color.b = 1.0

    for _, _, x, y, _ in rays[: self.candidate_downsample]:
        p = Point()
        p.x = float(x)
        p.y = float(y)
        p.z = 0.05
        m.points.append(p)

    self.target_cluster_pub.publish(m)

def publish_top_candidates(self, candidates: List[dict]) -> None:
    m = self.make_marker("nav_top_candidates", 0, Marker.SPHERE_LIST)
    m.scale.x = 0.12
    m.scale.y = 0.12
    m.scale.z = 0.12
    m.color.a = 1.0
    m.color.r = 1.0
    m.color.g = 0.55
    m.color.b = 0.0

    for item in candidates:
        x, y = item["target_xy"]
        p = Point()
        p.x = float(x)
        p.y = float(y)
        p.z = 0.10
        m.points.append(p)

    self.top_candidates_pub.publish(m)

def publish_target_marker(self, x: float, y: float) -> None:
    m = self.make_marker("nav_target_marker", 0, Marker.ARROW)

    start = Point()
    start.x = 0.0
    start.y = 0.0
    start.z = 0.12

    end = Point()
    end.x = float(x)
    end.y = float(y)
    end.z = 0.12

    m.points.append(start)
    m.points.append(end)

    m.scale.x = 0.045
    m.scale.y = 0.14
    m.scale.z = 0.22

    m.color.a = 1.0
    m.color.r = 1.0
    m.color.g = 0.0
    m.color.b = 1.0

    self.target_marker_pub.publish(m)

def delete_target_marker(self) -> None:
    m = self.make_marker("nav_target_marker", 0, Marker.ARROW)

```

```

        m.action = Marker.DELETE
        self.target_marker_pub.publish(m)

def publish_locked_target_marker(self, x: float, y: float) -> None:
    m = self.make_marker("nav_locked_target_marker", 0, Marker.ARROW)

    start = Point()
    start.x = 0.0
    start.y = 0.0
    start.z = 0.20

    end = Point()
    end.x = float(x)
    end.y = float(y)
    end.z = 0.20

    m.points.append(start)
    m.points.append(end)

    m.scale.x = 0.060
    m.scale.y = 0.18
    m.scale.z = 0.28

    m.color.a = 1.0
    m.color.r = 1.0
    m.color.g = 1.0
    m.color.b = 0.0

    self.locked_target_marker_pub.publish(m)

def delete_locked_target_marker(self) -> None:
    m = self.make_marker("nav_locked_target_marker", 0, Marker.ARROW)
    m.action = Marker.DELETE
    self.locked_target_marker_pub.publish(m)

def publish_fov_bounds(self) -> None:
    fov_min = math.radians(-60.0)
    fov_max = math.radians(60.0)

    length = 2.0
    z = 0.08

    rays = self.make_marker("nav_fov_bounds_marker", 0, Marker.LINE_LIST)
    rays.scale.x = 0.055
    rays.color.a = 1.0
    rays.color.r = 0.0
    rays.color.g = 1.0
    rays.color.b = 1.0

    for a in [fov_min, 0.0, fov_max]:
        p0 = Point()
        p0.x = 0.0
        p0.y = 0.0
        p0.z = z

        p1 = Point()
        p1.x = float(length * math.cos(a))
        p1.y = float(length * math.sin(a))
        p1.z = z

        rays.points.append(p0)
        rays.points.append(p1)

    self.fov_bounds_pub.publish(rays)

    arc = self.make_marker("nav_fov_bounds_marker", 1, Marker.LINE_STRIP)
    arc.scale.x = 0.050
    arc.color.a = 1.0
    arc.color.r = 0.0
    arc.color.g = 1.0
    arc.color.b = 1.0

    num_arc_points = 41

    for i in range(num_arc_points):
        t = i / float(num_arc_points - 1)
        a = fov_min + t * (fov_max - fov_min)

        p = Point()
        p.x = float(length * math.cos(a))
        p.y = float(length * math.sin(a))
        p.z = z

        arc.points.append(p)

    self.fov_bounds_pub.publish(arc)

    fan = self.make_marker("nav_fov_bounds_marker", 2, Marker.TRIANGLE_LIST)
    fan.scale.x = 1.0
    fan.scale.y = 1.0
    fan.scale.z = 1.0
    fan.color.a = 0.18
    fan.color.r = 0.0
    fan.color.g = 1.0
    fan.color.b = 1.0

    num_triangles = 40

```

```

for i in range(num_triangles):
    a0 = fov_min + (i / float(num_triangles)) * (fov_max - fov_min)
    a1 = fov_min + ((i + 1) / float(num_triangles)) * (fov_max - fov_min)

    center = Point()
    center.x = 0.0
    center.y = 0.0
    center.z = z - 0.01

    p0 = Point()
    p0.x = float(length * math.cos(a0))
    p0.y = float(length * math.sin(a0))
    p0.z = z - 0.01

    p1 = Point()
    p1.x = float(length * math.cos(a1))
    p1.y = float(length * math.sin(a1))
    p1.z = z - 0.01

    fan.points.append(center)
    fan.points.append(p0)
    fan.points.append(p1)

self.fov_bounds_pub.publish(fan)

def main(args=None) -> None:
    rclpy.init(args=args)
    node = ReactiveNav()

    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        node.publish_stop()
        node.publish_zero_pwm_lock()
        node.destroy_node()
        rclpy.shutdown()

if __name__ == "__main__":
    main()

```

7. run_robot.sh

```
#!/usr/bin/env bash

set -e

# =====
# run_robot
# Clean launch script for robot mapping/navigation
#
# Fix included:
# - Suppresses noisy sllidar / tf / slam logs
# - Creates a corrected /scan topic by rotating LiDAR scan data
#   180 degrees in the LaserScan message itself
# - Uses yaw = 0 for the LiDAR transform
# - Starts robot nodes in a controlled order
# =====

# -----
# User settings
# -----

WORKSPACE="$HOME/ros2_ws"

# Update these if your package/executable names are different
REACTIVE_PKG="reactive_nav"
REACTIVE_EXE="reactive_navigator"

MOTOR_PKG="motor_interface"
MOTOR_EXE="motor_interface"

# LiDAR settings
LIDAR_PORT="/dev/ttyUSB0"
LIDAR_BAUD="115200"

# Raw scan from sllidar
RAW_SCAN_TOPIC="/scan_raw"

# Corrected scan used by the rest of the robot
CORRECTED_SCAN_TOPIC="/scan"

# Frames
BASE_FRAME="base_link"
LIDAR_FRAME="laser"

# LiDAR pose relative to base_link
# Keep yaw = 0 because we are correcting the scan data directly.
LIDAR_X="0.0"
LIDAR_Y="0.0"
LIDAR_Z="0.12"
LIDAR_ROLL="0.0"
LIDAR_PITCH="0.0"
LIDAR_YAW="0.0"

# SLAM config
SLAM_PARAMS="$WORKSPACE/src/slam_toolbox/config/mapper_params_online_async.yaml"

# Log directory
LOG_DIR="$HOME/robot_logs"
mkdir -p "$LOG_DIR"

# -----
# Helper functions
# -----

cleanup() {
    echo ""
    echo "[run_robot] Shutting down robot stack..."

    pkill -f "sllidar_node" 2>/dev/null || true
    pkill -f "static_transform_publisher" 2>/dev/null || true
    pkill -f "async_slam_toolbox_node" 2>/dev/null || true
    pkill -f "scan_180_corrector.py" 2>/dev/null || true
    pkill -f "$REACTIVE_EXE" 2>/dev/null || true
    pkill -f "$MOTOR_EXE" 2>/dev/null || true

    echo "[run_robot] Done."
}

trap cleanup EXIT

wait_for_topic() {
    local topic_name="$1"
    local timeout_sec="$2"
    local elapsed=0

    echo "[run_robot] Waiting for topic $topic_name..."

    while [ "$elapsed" -lt "$timeout_sec" ]; do
        if ros2 topic list 2>/dev/null | grep -qx "$topic_name"; then
            echo "[run_robot] Found $topic_name."
            return 0
        fi

        sleep 1
        elapsed=$((elapsed + 1))
    done
}
```

```

done

echo "[run_robot] ERROR: Timed out waiting for $topic_name."
return 1
}

print_build_time() {
    local pkg="$1"
    local exe="$2"

    local path
    path="$(ros2 pkg prefix "$pkg" 2>/dev/null || true)"

    if [ -z "$path" ]; then
        echo "[run_robot] $pkg not found."
        return
    fi

    local full_exe="$path/lib/$pkg/$exe"

    if [ -f "$full_exe" ]; then
        echo "[run_robot] $pkg/$exe built: $(stat -c '%y' "$full_exe")"
    else
        echo "[run_robot] Could not find executable: $full_exe"
    fi
}

# -----
# Start clean
# -----

echo "[run_robot] Cleaning up old robot processes..."

pkill -f "sllidar_node" 2>/dev/null || true
pkill -f "static_transform_publisher" 2>/dev/null || true
pkill -f "async_slam_toolbox_node" 2>/dev/null || true
pkill -f "scan_180_corrector.py" 2>/dev/null || true
pkill -f "$REACTIVE_EXE" 2>/dev/null || true
pkill -f "$MOTOR_EXE" 2>/dev/null || true

sleep 2

# -----
# Source ROS workspace
# -----

if [ -f "/opt/ros/humble/setup.bash" ]; then
    source /opt/ros/humble/setup.bash
else
    echo "[run_robot] ERROR: Could not find /opt/ros/humble/setup.bash"
    exit 1
fi

if [ -f "$WORKSPACE/install/setup.bash" ]; then
    source "$WORKSPACE/install/setup.bash"
else
    echo "[run_robot] ERROR: Could not find $WORKSPACE/install/setup.bash"
    exit 1
fi

echo "[run_robot] ROS environment loaded."

print_build_time "$REACTIVE_PKG" "$REACTIVE_EXE"
print_build_time "$MOTOR_PKG" "$MOTOR_EXE"

# -----
# Check LiDAR port
# -----

if [ ! -e "$LIDAR_PORT" ]; then
    echo "[run_robot] ERROR: LiDAR port $LIDAR_PORT does not exist."
    echo "[run_robot] Available serial devices:"
    ls -l /dev/ttyUSB* /dev/serial/by-id/* 2>/dev/null || true
    exit 1
fi

echo "[run_robot] Using LiDAR port: $LIDAR_PORT"

# -----
# Create temporary scan corrector node
# -----

SCAN_CORRECTOR="/tmp/scan_180_corrector.py"

cat > "$SCAN_CORRECTOR" << 'EOF'

```


8. script_8.py

```
#!/usr/bin/env python3

import math
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import LaserScan

class Scan180Corrector(Node):
    def __init__(self):
        super().__init__('scan_180_corrector')

        self.declare_parameter('input_topic', '/scan_raw')
        self.declare_parameter('output_topic', '/scan')
        self.declare_parameter('frame_id', 'laser')

        input_topic = self.get_parameter('input_topic').value
        output_topic = self.get_parameter('output_topic').value
        self.frame_id = self.get_parameter('frame_id').value

        self.sub = self.create_subscription(
            LaserScan,
            input_topic,
            self.scan_callback,
            10
        )

        self.pub = self.create_publisher(
            LaserScan,
            output_topic,
            10
        )

        self.get_logger().info(
            f'Correcting LaserScan by 180 degrees: {input_topic} -> {output_topic}'
        )

    def scan_callback(self, msg):
        out = LaserScan()

        out.header = msg.header
        out.header.frame_id = self.frame_id

        out.angle_min = self.normalize_angle(msg.angle_min + math.pi)
        out.angle_max = self.normalize_angle(msg.angle_max + math.pi)

        # If adding pi makes angle_min greater than angle_max, keep the scan
        # continuous by shifting the interval back into a valid increasing range.
        if out.angle_min > out.angle_max:
            out.angle_min -= 2.0 * math.pi

        out.angle_increment = msg.angle_increment
        out.time_increment = msg.time_increment
        out.scan_time = msg.scan_time
        out.range_min = msg.range_min
        out.range_max = msg.range_max

        # Important:
        # Do NOT reverse the ranges here.
        # We are rotating the scan angles by 180 degrees while preserving
        # each range's angular index relationship.
        out.ranges = list(msg.ranges)
        out.intensities = list(msg.intensities)

        self.pub.publish(out)

    @staticmethod
    def normalize_angle(angle):
        while angle > math.pi:
            angle -= 2.0 * math.pi
        while angle <= -math.pi:
            angle += 2.0 * math.pi
        return angle

def main(args=None):
    rclpy.init(args=args)
    node = Scan180Corrector()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        node.destroy_node()
        rclpy.shutdown()

if __name__ == '__main__':
    main()
EOF

chmod +x "$SCAN_CORRECTOR"

# -----
```

```

# Start LiDAR driver quietly
# -----

echo "[run_robot] Starting SLLIDAR quietly on $RAW_SCAN_TOPIC..."

ros2 run sllidar_ros2 sllidar_node \
  --ros-args \
  -r __node:=sllidar_node \
  -r scan:="$RAW_SCAN_TOPIC" \
  -p serial_port:="$LIDAR_PORT" \
  -p serial_baudrate:="$LIDAR_BAUD" \
  --log-level ERROR \
  > "$LOG_DIR/sllidar.log" 2>&1 &

sleep 3

wait_for_topic "$RAW_SCAN_TOPIC" 15

# -----
# Start corrected scan publisher
# -----

echo "[run_robot] Starting 180-degree LaserScan corrector..."

python3 "$SCAN_CORRECTOR" \
  --ros-args \
  -p input_topic:="$RAW_SCAN_TOPIC" \
  -p output_topic:="$CORRECTED_SCAN_TOPIC" \
  -p frame_id:="$LIDAR_FRAME" \
  --log-level WARN \
  > "$LOG_DIR/scan_corrector.log" 2>&1 &

sleep 2

wait_for_topic "$CORRECTED_SCAN_TOPIC" 15

# -----
# Static transform
# -----

echo "[run_robot] Starting static transform: $BASE_FRAME -> $LIDAR_FRAME"

ros2 run tf2_ros static_transform_publisher \
  "$LIDAR_X" "$LIDAR_Y" "$LIDAR_Z" \
  "$LIDAR_ROLL" "$LIDAR_PITCH" "$LIDAR_YAW" \
  "$BASE_FRAME" "$LIDAR_FRAME" \
  --ros-args --log-level ERROR \
  > "$LOG_DIR/static_tf.log" 2>&1 &

sleep 1

# -----
# Start SLAM
# -----

echo "[run_robot] Starting SLAM toolbox..."

if [ -f "$SLAM_PARAMS" ]; then
  ros2 launch slam_toolbox online_async_launch.py \
    slam_params_file:="$SLAM_PARAMS" \
    use_sim_time:=false \
    > "$LOG_DIR/slam_toolbox.log" 2>&1 &
else
  echo "[run_robot] WARNING: SLAM params file not found:"
  echo "[run_robot] $SLAM_PARAMS"
  echo "[run_robot] Starting SLAM toolbox with default params."

  ros2 launch slam_toolbox online_async_launch.py \
    use_sim_time:=false \
    > "$LOG_DIR/slam_toolbox.log" 2>&1 &
fi

sleep 4

# -----
# Start reactive navigator
# -----

echo "[run_robot] Starting reactive navigator..."

ros2 run "$REACTIVE_PKG" "$REACTIVE_EXE" \
  --ros-args \
  -p scan_topic:="$CORRECTED_SCAN_TOPIC" \
  -p lidar_scan_offset_deg:=0.0 \
  -p steering_sign:=1.0 \
  -p linear_sign:=1.0 \
  --log-level INFO \
  > "$LOG_DIR/reactive_navigator.log" 2>&1 &

sleep 2

# -----
# Start motor interface
# -----

echo "[run_robot] Starting motor interface..."
ros2 run "$MOTOR_PKG" "$MOTOR_EXE" \

```

```

--ros-args --log-level INFO \
> "$LOG_DIR/motor_interface.log" 2>&1 &

sleep 2

# -----
# Summary
# -----

echo ""
echo "[run_robot] Robot stack started."
echo "[run_robot] Raw LiDAR topic:      $RAW_SCAN_TOPIC"
echo "[run_robot] Corrected scan topic: $CORRECTED_SCAN_TOPIC"
echo "[run_robot] LiDAR TF yaw:        $LIDAR_YAW rad"
echo "[run_robot] LiDAR scan offset:    applied in LaserScan data, not TF"
echo "[run_robot] Logs saved in:       $LOG_DIR"
echo ""
echo "[run_robot] To check scans:"
echo "  ros2 topic echo $CORRECTED_SCAN_TOPIC"
echo ""
echo "[run_robot] To stop, press Ctrl+C."
echo ""

while true; do
  sleep 1
done

```